An optimisation approach for supply chain procurement transport operational planning with 3D full-truckload constraints and returnable transport items

This paper proposes a mixed integer linear programming model, dubbed 3D-SCPTOP, for procurement transport planning in manufacturing supply chains, integrating the procurement, inventory and transport decisions of an assembly plant and its supplier with three-dimensional full-truckload constraints and the closed-loop management of returnable transport items. Because the placement subproblem is NP-hard and the compact model carries one copy of it per truck, direction and period, we reformulate the problem by Dantzig–Wolfe decomposition, taking the complete loaded truck as the unit of decision. The decomposition is derived structurally from a model stated over trucks in which *no set of loading configurations appears at all*: deleting the quantifier over (direction, truck, period) from its block rows leaves the pricing problem, the trucks of a direction and period are anonymous and are aggregated into counts, and the master couples to the geometry through nothing but two aggregate-flow definitions and one truck count per direction and period. Every geometric, stack-height, crush, payload and full-truckload constraint lives inside one pricing problem per direction. A column is a geometrically realisable truck load with its layout retained, so the per-stack crush cap is evaluated on the product mix actually loaded rather than on a worst-case bound; a plan assembled from such columns is executable by construction, and each dispatched truck is re-verified independently of the search and of the oracle that built the column. The master is solved by branch and price, with a two-level branch and bound over the fleet vector and a flow-first relax-and-fix construction as primal heuristic whose integer programmes carry necessary conditions of placeability as rows, and because every placeable load is volumetrically feasible, a certified bound of the volumetric relaxation is a valid lower bound for the 3D problem. The method is assessed in two stages. Against the compact formulation on the volumetric problem, where a reference optimum exists, it returns a plan 0.0001% above a proved optimum at the same fleet of forty-five trucks with a certified bound 0.0113% below that optimum, and it agrees with the compact model's incumbent within 0.31% on four further instances on which that model exhausts its memory budget; the columns of that stage carry no layout, so its plans are *not* executable. On the 3D problem, where the compact model returns no incumbent beyond five periods, the method returns verified plans on all five rungs of a ladder reaching twenty-four products and twenty periods, from 12 to 156 trucks within budgets of 900 to 3 600 s, with provably minimum fleets through twelve periods. Certified gaps run from 0.0075% to 0.163% on the first three rungs and are 14.5% and 10.7% on the last two: small where the volumetric relaxation is nearly exact, large where a per-stack crush limit halves a container family's layer count, and which of the two regimes an instance falls in is computable from the container catalogue before any model is solved. On the ladder instances $I_16_15_2_90_k25$ and $I_24_20_2_90_k50$, a volumetric model's plan prescribes loads that are *proved* impossible to place in a trailer, and re-planning its trucks while keeping its delivery quantities needs more trucks than the method plans directly. A volumetric transport capacity, which most of the procurement transport literature assumes, is therefore not a conservative simplification — where it errs it errs by prescribing loads that do not fit, discovered on the loading dock and not in the model — and on the industrial container catalogue the error runs one way: of its fifteen types *none* is credited below an analytic count of what a trailer can hold, nine sit exactly at it, and four are over-credited by a fifth or more. That test needs only the catalogue and the trailer, so it is a property of the catalogue to be checked rather than assumed.

Keywords— Procurement, inventory, transport planning, full-truckload, 3D container loading

1 Introduction

Optimizing the use of resources in manufacturing supply chains is crucial for boosting economic growth, improving social well-being, and minimizing environmental impact. Integrating transport, inventory, and production processes is a fundamental characteristic of supply chains (Stadtler, Kilger, and Meyr 2015) because joint decisions will improve supply chain performance(Díaz-Madroñero, Mula, and Peidro 2017).

Procurement refers to the process of acquiring goods and services in a supply chain (Chopra and Meindl 2015). One of the main goals of procurement is to obtain raw materials, parts or finished goods for a factory or distribution centre from a supplier. As mentioned in (Fleischmann, Kopfer, and Sürie 2015), the responsibility for transport planning usually lies with the supplier. However, sometimes the manufacturer makes transport decisions of its supplies in, for example, the automotive industry. Furthermore, an essential asset in the automotive supply chain is not considered in traditional material requirement planning (MRP): returnable containers or returnable transport items (RTI), which are used as tertiary packaging. RTIs enable the storage and transportation of RM and FG through the supply chain. Usually, the manufacturing plant supplies empty RTIs to its supplier, while the supplier delivers full RTIs with the FG or RM. Therefore, during the procurement process, transport and inventory decisions should be integrated, including RTI management. Here, we focus on optimising the operational decisions made during procurement transport planning.

According to (Díaz-Madroñero, Peidro, and Mula 2014), using algorithm-based methods for procurement transport planning is more effective than relying on manual approaches based on planners' intuition. Manual planning can be short-sighted and lead to less efficient outcomes. This becomes even more significant when optimizing procurement transport tasks that aim to reduce the number of trucks and inventory levels, especially under 3D loading constraints where the precise placement of containers within trucks must also be determined. Furthermore, the capacity constraints used in the scientific literature on procurement transport planning are based mainly on freight volume or the total number of loaded product units (Hashemi-Amiri, Ghorbani, and Ji 2023) and (Díaz-Madroñero, Peidro, and Mula 2014). The current research seeks to address this gap by introducing a new mixed-integer linear programming (MILP) model. This model uniquely combines procurement transportation with considerations for 3D loading, inventory management, and RTI management. Hereafter, we refer to this model formulation as 3D supply chain procurement transport operational planning (3D-SCPTOP). Additionally, a branch-and-price procedure (BP-3D-SCPTOP) is developed to solve it.

In this paper, we present four main contributions, listed in the order of the evidence behind them. (i) We formulate a novel MILP model, 3D-SCPTOP, integrating procurement transport, RTI management and inventory decisions with stack-based full-truck loading constraints. (ii) We reformulate it by Dantzig–Wolfe decomposition over geometrically realisable truck loads, deriving the decomposition structurally — the model is written over trucks, so deleting the quantifier over (direction, truck, period) from its block rows is the pricing problem — and obtaining a master in which *no loading configuration appears at all*: the master couples to the geometry through two aggregate-flow definitions and one truck count per direction and period, and the pricing problem of a direction is the compact model's own block with the quantifier over trucks removed. We solve it by a branch and bound over the fleet vector with column generation over that mixed integer pricing problem and a relax-and-fix construction as primal heuristic, giving BP-3D-SCPTOP, and we show that a certified bound of the volumetric relaxation is a valid lower bound for the 3D problem. (iii) We establish the method's accuracy where a reference optimum exists — on the volumetric problem, against the compact formulation, which it matches to 0.0001% of a proved optimum (Section 9.2) — and then show that it keeps returning verified, executable plans and certified bounds in the 3D regime beyond which the compact model returns no incumbent at all. (iv) We show by independent placeability proof that a volumetric representation of truck capacity, as used in the procurement transport literature, yields plans that are cheaper on paper but cannot be loaded in practice, and we quantify what repairing them costs.

Readers are referred to the typology by (Wäscher, Haußner, and Schumann 2007) for a global view of container loading problems, where the different dimensions of the loading problem are described. Furthermore, (Silva, Toffolo, and Wauters 2019) summarise the exact methods used in the literature to solve container loading problems, and (X. Zhao et al. 2016) explore other solution approaches in the literature. Readers are also referred to the literature reviews of (Mosca, Vidyarthi, and Satir 2019) and (Díaz-Madroñero, Peidro, and Mula 2015), which focus more on integrating inventory decisions and transport. Here, we jointly address container loading, procurement transport and inventory decisions.

The paper is organised as follows. Section 2 provides a literature review related to the problem under

study. Section 3 presents the problem description. Section 4 describes the 3D-SCPTOP MILP formulation. Sections 5–8 develop the branch and price solution approach, BP-3D-SCPTOP, together with its pricing subproblem and bounding arguments. Section 9 reports the computational experiments. Finally, Section 10 concludes and puts forward further research lines.

2 Literature Review

The goal of this research is to integrate procurement transport operational planning decisions with 3D loading restrictions and RTI packaging. Based on the authors' research, no previous work has been identified that meets this criterion. Therefore, three independent, but sometimes overlapping, knowledge bodies have been reviewed: procurement transport and inventory planning, container loading problems and RTI literature.

2.1 Procurement transport and inventory planning

Models that integrate transport and inventory costs during the procurement process are widely studied in the literature. As noted in (Langley et al. 2020), transportation facilitates the movement of raw materials and finished products between suppliers, warehouses, manufacturers, and retailers. Therefore, transport processes play a crucial role in supply chains. (Fleischmann, Kopfer, and Sürie 2015) distinguish between two main types of transportation within a supply chain: "procurement transport," which involves moving materials from suppliers to manufacturing sites, and the "distribution of goods," which concerns delivering finished products from factories to customers. This article focuses specifically on addressing the challenges related to procurement transportation.

There are two main dimensions of procurement transport problems according to the supply chain configuration and the applied transport method. Regarding supply chain configurations (Mosca, Vidyarthi, and Satir 2019): (i) single source to a single destination (SS); (ii) multiple sources to a single destination (MS); (iii) single source to multiple destinations (SM); (iv) multiple sources to multiple destinations (MM). Source components determine the number of supplier locations that are taken into account, while destination components denote the number of supplied manufacturers. Additionally, different transport methods are identified in (Díaz-Madroñero, Peidro, and Mula 2014):milk run, less-than-truckload (LTL), full-truckload (FTL) . We also analyze the formulation of the transportation capacity constraints in the scientific literature.

(Berman and Wang 2006) address the problem of optimising distribution strategies to deliver products from suppliers to plants, whose aim is to minimise transport, pipeline and plant inventory costs by using cross docks. A mixed integer non-linear programming (MINLP) formulation, a Lagrangian relaxation heuristic, a greedy heuristic, and a branch-and-bound algorithm are proposed to model and solve the problem.

(Sancak and Salman 2011) present a MILP model to optimise procurement transport decisions in a SS supply chain configuration using LTL and FTL transport methods. The objective is to minimise transport and inventory holding costs, while fulfilling production plan requirements on a multiperiod planning horizon. The study explores the possibility of delaying LTL shipments to the subsequent time period, and utilising safety stocks whenever necessary, in a stochastic environment by employing data from a bus manufacturer. The findings demonstrate that delaying shipments leads to reduced holding and transport costs with a minimal stockout risk.

(Schöneberg, Koberstein, and Suhl 2010) examine the concept of delivery profiles, which define the delivery frequency at a fixed rate, offering structured planning, minimizing management effort, and improving coordination in logistics. In this context, FTL transport is used when the goods' volume exceeds a vehicle's capacity, making direct shipment possible. LTL is applied when the volume is insufficient to fill a truck, and the goods are first sent to a consolidation center before reaching the final destination. In (Schöneberg, Koberstein, and Suhl 2013), the proposed model accounts for demand uncertainty.

(Díaz-Madroñero, Mula, and Peidro 2014) present a fuzzy multi-objective integer linear programming (FMOILP) model that focuses on operational procurement transport planning in an automotive supply chain, which involves a single source and a single destination while utilising FTL. The developed model aims to minimise the number of trucks employed for procurement transport and the inventory level by considering the uncertainties related to transport capacity levels.

(Hein and Almeder 2016) investigate the benefits of integrated procurement transport and production planning for supply and production processes by comparing a sequential approach, where production is firstly determined, followed by procurement transport decisions, plus an integrated approach in which decisions are jointly made. Numerical experiments reveal that the integrated planning approach leads to substantial cost savings.

(Díaz-Madroñero, Mula, and Peidro 2017) present an novel MILP model that integrates procurement, production and transport planning decisions within manufacturing systems. This model extends beyond traditional material requirement planning (MRP) systems by incorporating procurement transport decisions from multiple sources and a single destination into a unified optimization framework. It considers the capacities of both material and production resources, along with procurement transport planning decisions, by employing various shipping modes such as FTL, LTL, and milk-run.

(Chitsaz, Cordeau, and Jans 2019) address integrating production, inventory and procurement transport into a manufacturing context by defining this problem as an assembly routing problem (ARP). It introduces a novel MILP model for the ARP and proposes a three-phase decomposition matheuristic for its solution.

(Baller et al. 2022) examine the optimization of procurement transport in an automotive factory, focusing on first-tier suppliers at an aggregate level across multiple sources (570) to a single destination, using FTL, LTL, and courier express service (CES) as transportation methods. The proposed MILP approach includes a cyclic inventory constraint, truck utilization, and storage limitations.

(Hashemi-Amiri, Ghorbani, and Ji 2023) propose an integrated approach to optimize a perishable food supply chain. The model combines supplier selection, procurement and distribution transport, production scheduling, and vehicle routing to improve supply chain efficiency and resilience. It handles demand uncertainties through a distributionally robust chance-constrained model. The model is reformulated as MILP by utilizing duality and linearization techniques.

2.2 Procurement Transport with RTIs

Some models further integrate the procurement and delivery of RTIs into the procurement transport literature. In this sense, RTIs are procured to the supplier, who loads the items and sends them to customer. We extend the SC definitions previously defined to contemplate the RTIs management strategies.

(Soysal et al. 2023) develop a tactical-level, multi-period closed-loop Vendor-Managed Inventory (VMI) system for RTIs in an agro-food SC. The SM SC integrates forward flows of full RTIs from a single vendor to multiple customers and reverse flows of empties back to the vendor for cleaning . All routing and inventory decisions are made centrally by the vendor under full information sharing. Sustainable plans in terms of amounts of energy spent and empty crates collected are devised and evaluated.

(Guo et al. 2024) model two-tier Container Management Centre (CMC) network . Empty RTIs are dispatched from central CMC hubs to upstream suppliers, loaded with products, carried by truck to downstream customers, then returned empty to regional CMCs for cleaning and storage before being shipped back to central hubs. A neural network is used for demand forecasting. A MILP is proposed that maximises total profit by optimally determining transport costs and the penalties applied to late RTIs returns.

(Y. Zhang et al. 2024) formulate a perishable-food inventory-routing MILP with multiple RTI types in a SM supply chain, integrating procurement of products and pickup of both filled and empty RTIs. Decisions are made on a operational level, optimally determining per period the quantities to pack, deliver, pick up, purchase and lose, as well as the vehicle routes in order to maximise the total profit of the SC that is equal to the total quality-dependent revenue minus the total costs involved, including inventory holding costs, product cost, transportation costs, and RTI purchasing and loss costs. The model is solved exactly via CPLEX solver and approximately via a tailored Kernel Search matheuristic.

2.3 Container loading

Container loading problems encompass a subset of geometric assignment problems known as cutting and packing ($C\&P$) problems. They revolve around the allocation of n-dimensional orthotope items in larger n-dimensional orthotope containers. Here the aim is to optimise an objective function while taking into account two key geometric constraints: complete containment of items inside containers and lack of item overlap. A loading pattern serves as a formal representation of a solution to such an assignment problem. Dyckhoff introduced in (Dyckhoff 1990) a typology that unifies $C\&P$ problems using four different criteria: dimensionality of space, the objective of assignment, containers types and box type. Consequently, (Wäscher, Haußner, and Schumann 2007) develop a new typology that has become the standard in the $C\&P$ literature. We briefly describe the typology used in (Wäscher, Haußner, and Schumann 2007) based on five different criteria: (i) dimensionality of space; (ii) kind of assignment (input minimisation of containers for all small items and loading patterns; output maximisation to pack a subset of small items with limited container availability;) (iii) assortment of small item by distinguishing if they are either a weakly (or strongly) heterogeneous assortment or identical small items ; (iv) assortment of large objects (if it is either several large objects or one large object); (v) shape of small items (circular or of another shape).

The first mathematical programming model in a scientific journal to describe the container-loading problem was developed in (C. S. Chen, Lee, and Shen 1995), where the authors introduced an MILP model to address the 3D input minimisation problem by considering rotations and containers of varying dimensions. However, it does not specify the use of weakly or strongly heterogeneous boxes or containers. Additionally, real-world constraints are modelled like, for example, weight distribution.

(Junqueira, Morabito, and Sato Yamashita 2012b) propose an alternative MILP formulation to (C. S. Chen, Lee, and Shen 1995) using binary encoding for box position, which utilises normal patterns to reduce possible box coordinates by considering multiples of boxes dimensions and ensuring optimal solutions. Additionally, loading and cargo constraints are developed to ensure the static and dynamic stability of the cargo. Boxes must be supported by other box top faces or the container floor. The same authors integrate further loading restrictions in (Junqueira, Morabito, and Sato Yamashita 2012a), where they integrate multidrop constraints, which address the challenge of packing a truck with boxes for different customers at various destinations. The goal is to plan the container's loading to minimise the time spent unloading and reloading boxes at each destination.

(W. Zhu and Lim 2012) devise a tree search-based method to solve SLOPP and SKP with 3D constraints. Their approach involves evaluating free spaces by employing a look-ahead tree search and selecting optimal placements based on the best free space-block pairs.

(Gonçalves and Resende 2013) develop a biased random-key genetic algorithm (BRKGA) designed to solve SBSBPP. The algorithm addresses geometric constraints and introduces a maximal-space representation to handle available free spaces in bins. A novel placement procedure is hybridised with the BRKGA, which optimises the order of box packing and the parameters utilised by the placement procedure to offer an effective solution approach to an NP hard problem.

(Bortfeldt and Wäscher 2013) identify constraints for $C\&P$, including item-related constraints (orientation constraints, load bearing, loading priorities), container-related constraints (weight distribution and weight limit), and cargo-related constraints (complete-shipment constraints) and load-related constraints (complexity constraints or stability constraints). The study also highlights the achievements and deficits in container loading research that pertain to these constraints.

(Paquay, Schyns, and Limbourg 2016) propose an MILP model for the MBSBPP. One noteworthy constraint is the assumption that containers are truncated parallelepiped instead of rectangular boxes. They also consider constraints, such as box rotation, container load bearing, weight distribution, stability and fragility of items.

(Mahvash, Awasthi, and Chauhan 2018) introduce a novel resolution procedure for the SBSBPP through the column generation technique, where the columns that are generated are the loading configurations of a single bin. For the pricing develop problem they develop a heuristic to find new columns with a negative reduced cost. This approach not only produces better results than the previous articles on the literature, but also gives new lower bounds.

(Silva, Toffolo, and Wauters 2019) study SSLOP models with 3D constraints. These authors compare the different approaches that exist in the literature for model packing problems based on two fundamental approaches: discretised formulation (Junqueira, Oliveira, et al. 2013) and continuous formulation (C. S. Chen, Lee, and Shen 1995) with subsequent modifications. They also indicate that discretised formulation(Junqueira, Morabito, and Sato Yamashita 2012a) performs the best for instances with similar sized boxes. However, it encounters memory limitations for large instances. For other sets of instances, the adapted formulations of (C. S. Chen, Lee, and Shen 1995) and (Paquay, Schyns, and Limbourg 2016) outperform other adaptations. Further research topics suggest including the design of new formulations by considering the limited approaches in the literature.

(Kurpel et al. 2020) introduce techniques and exact approaches to solve input minimisation and output maximisation problems for container-loading problems. Their methods consider practical constraints, such as orientation, separation of boxes and vertical stability. A set partitioning method is also employed in order to obtain upper bounds in a container-loading input minimisation problems. The authors adapt and evaluate four discretisation methods found in the existing literature and incorporate mathematical formulations to address the aforementioned constraints.

(Alonso, Alvarez-Valdes, and Parreño 2020) develop an algorithm by the greedy randomised adaptive search procedure (GRASP) to tackle a practical container-loading problem with time-sensitive constraints. Their approach addresses dynamic stability constraints while stacking products on pallets and loading them onto trucks during different time periods. Their approach provides high-quality solutions and surpasses an integer programming formulation in efficiency and effectiveness terms.

(Ali et al. 2022) conduct a comprehensive literature review by identifying constraints and solution methods for offline problems with full knowledge of the items to be packed and those tackling the online problems, where items are packed immediately when they arrive, and no prior knowledge of an item is known. The

study also identifies and classifies solution approaches for offline packing into six categories: exact methods, constructive heuristics; local search and metaheuristic methods; approximation algorithms; machine learning and hyper-heuristic methods; tree search methods. (Harrath 2022) proposed a thee stage heuristic based on layer generation to find good quality solutions the SBSPP problem for large instance up to 993 bins and 200000 items taking into account load balancing constraints.

(Tresca et al. 2022) develop a matheuristic to solve the SBSBPP that includes items with different IDs and different categories, to be delivered to different clients. The algorithm creates monoitem, monocategory, and mixed layers which then are used to compute the final loading configurations of the bins.

(Li, M. Chen, and Huo 2022) develope a hybrid adaptaive large neighborhood search algoritm to solve the multiple container loading problem, which is tries pack a given set of items with the minimum number of containers while maximazing the space utilization of them. Therfore, it is both a MBSBPP and a MSSCSP. Their approach takes into account some practical applications such as supsension constraints, rotation and weight limit.

(M. Chen, Huo, and Duan 2023) present a new approach to the 3D-BPP, introducing the 3D-BSDPP which focuses on optimizing both the design of bin sizes and the packing process. A novel Hybrid Biogeography-Based Optimization (HBBO) algorithm is proposed, integrating differential evolution and a local search strategy for enhanced performance.

One noteworthy trend in $C\&P$ is integrating loading problems with routing. This is known as capacitated vehicle-routing problem with 3D or 2D loading constraints (3L-CVRP or 2L-CVRP). (Junqueira, Morabito, and Sato Yamashita 2012b) present a MILP model with binary encodings for 3L-CVRP by including loading, cargo and multidrop constraints. The combination of these two NP-hard problems requires using heuristics and metaheuristics for real-world instances, as seen in (Dominguez et al. 2016), (Alonso, Martinez-Sykora, et al. 2022) and (Wei et al. 2015). Another noteworthy trend is to use reinforcement learning techniques to solve $C\&P$, as in (Que, Yang, and D. Zhang 2023) and (H. Zhao, C. Zhu, and Huang 2022).

2.4 Research Gaps & Methodology

Here, we discuss the main differences between the 3D-SCPTOP model proposal and those reviewed by specifically focusing on procurement transport. The key distinction of the 3D-SCPTOP model compared to the existing literature is the incorporation of 3D constraints and vertical stability constraints to model transport capacity with a view to providing more realistic solutions and reducing the likelihood of infeasible outcomes. Table 1 presents a comparison of inventory modelling, transport modelling, and supply chain configuration between the 3D-SCPTOP model proposal and those currently in the literature. Hence, the 3D-SCPTOP approach is based on a SS supply chain configuration, and takes into account the FTL transportation mode.

Article	SC Configuration	Decision level	Transportation Mode	Transportation Capacity Constraint	Transportation Cost	Inventory Capacity Restriction	RTIs	Inventory Costs	Safety Stocks	Modelling Framework
(Guo et al. 2024)	MM	TO	FTL	UH	X	X	S—NF	X		MINLP
(Berman and Wang 2006)	MM	T	FTL—LTL	V	X			X		MINLP
(Sancak and Salman 2011)		O	LTL—FTL	V	X			X	X	MILP
(Schöneberg, Koberstein, and Suhl 2010)	MS	T	FTL—LTL	V	X	X		X		MILP
(Schöneberg, Koberstein, and Suhl 2013)	MS	O	FTL—LTL	V	X	X		X		MILP
(Díaz-Madroñero, Mula, and Peidro 2014)	SS	O	FTL	UH	X			X	X	FMOILP
(Hein and Almeder 2016)	MS	O	MR	V	X			X		MILP
(Díaz-Madroñero, Mula, and Peidro 2017)	MS	T	FTL—LTL—MR	V	X	X		X	X	MILP
(Meyer and Amberg 2018)	MS	T—O	MR—FTL	V	X	X		X		MILP
(Chitsaz, Cordeau, and Jans 2019)	MS	O—T	MR	V	X	X		X		MILP
(Baller et al. 2022)	MS	O	MR—FTL—LTL—CES	V—W	X	X			X	MILP
(Hashemi-Amiri, Ghorbani, and Ji 2023)	MS	T	MR	V	X					CCP
3D-SCPTOP	SS	O	FTL	3D-Constraints	X	X		X	X	MILP
(Y. Zhang et al. 2024)	MS	TO	MR	UH	X	X	M—NF	X		ILP

Abbreviations. **T** (tactical), **O** Operational; **FTL** (full-truck load), **LTL** (less-than-truck load), **MR** (milk-run), **CES** (courier express service); **V** (volumetric), **W** (weight), **UH** Unit Holding; **CCP** chance-constrained programming

Table 1: Comparison of procurement transport models

On the other hand, the 3D-SCTOP model diverges from those proposed in the packaging literature in several aspects. Firstly, the loading problem takes into account different time periods. Therefore, the specific container (small items or boxes in the $C\&P$ literature) that we utilise are yet to be defined, as the choice of container depends on the inventory levels of each product. Also, the objective function depends on inventory and transport costs, which are inherently connected to the loading problem at hand. Thefore, we simultaneously solve an input minimisation and output maximisation problem. Both of these are a result of transport minimisation and inventory costs. Input minimisation comes because we want to find the minimum number of trucks needed. Meanwhile, output maximization comes from the FTL and the transport costs.

Article	Problem type	Resolution procedure	Orientation	Weight	Load Balance	Stability	Time-Depenent
(W. Zhu and Lim 2012)	SLOP—SKP	TSM	X	X			
(Gonçalves and Resende 2013)	SBSBPP	BRKGA	X	X		X	
(Paquay, Schyns, and Limbourg 2016)	SBSBPP	EM	X	X	X	X	
(Mahvash, Awasthi, and Chauhan 2018)	SBSBPP	CG	X	X			
(Alonso, Alvarez-Valdes, and Parreño 2020)	SSSCSP	GRASP	X	X	X	X	X
(Tresca et al. 2022)	SBSBPP	MH	X	X	X	X	
(Harrath 2022)	SBSBPP	H	X	X	X		
(Li, M. Chen, and Huo 2022)	SBSBPP—MSSCSP	HALNs	X	X		X	
(M. Chen, Huo, and Duan 2023)	BSDPP	HBBO	X	X			
3D-SCPTOP	SSSCSP & MILOPP	BP	X	X	X	X	X

Abbreviations. **T**SM (Tree search method), **BRKGA** Biased Randomized Jey Genetic Algorithm; **EM** (Exact Metods), **MH** (MatHeuristic), **CG** (Column Generation), **H** (Heuristic); **BP** (Branch and Price); **HCLP** (Hybrid Adaptive Large Neighborhood Search), **HBBO** (Hybrid Biogeography-Based Optimization)

Table 2: Comparison of C&P problems

Based on the literature review and the identified research gaps, our proposal is structured according to the steps depicted in Figure 1.

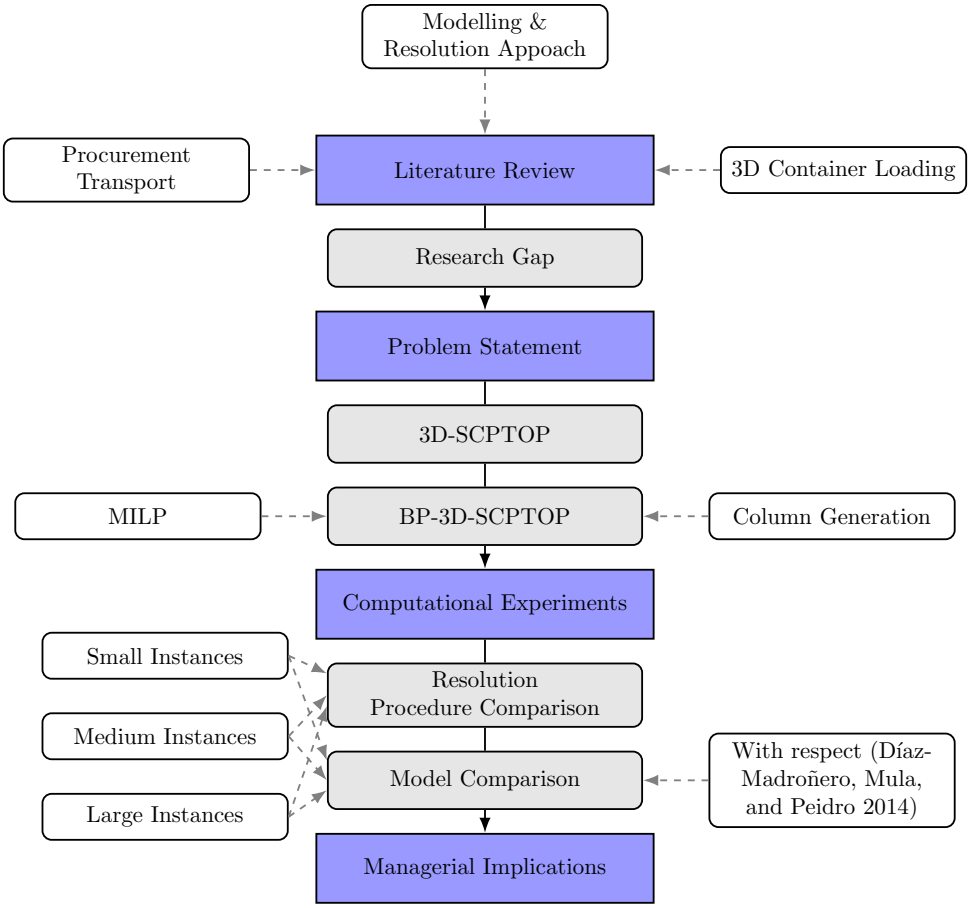


Figure 1: Research Methodology

3 Problem description

A plant assembles products supplied by a single supplier over a finite horizon. Every product travels in exactly one type of RTI: a rigid container, *erected* while it carries product and *folded* when empty, so one physical container occupies two different volumes depending on which way it is moving. The supplier packs its production into erected RTIs and ships them to the plant; the plant consumes the product, folds the containers and returns them. Both legs are run with identical trailers, every dispatch costs the same whatever

it carries, and a trailer may be dispatched only if it is full. A dispatch arrives lt periods after it leaves and is available at neither end in between, and each node must end every period holding enough stock to cover the next periods of its own activity. What has to be decided is, per direction and period, how many trucks go, what each carries and what consequently sits in stock; the cost is the number of dispatches plus the inventory held.

Two features separate this from a transport-and-inventory model. The two flows are coupled through the containers themselves — the plant can return only empties it has emptied and the supplier can fill only containers it has received — so a shortage on the return leg propagates into the delivery leg one lead time later. And the admissibility of a truck load is a three-dimensional packing condition rather than a capacity inequality: a load whose volume and weight both fit may still have no arrangement inside the trailer. Loading is represented through *stacks* of whole homogeneous layers rather than through the placement of individual RTIs, which makes the stacks self-supporting, so no vertical-stability constraint is needed.

4 The 3D-SCPTOP formulation

The model is written over *trucks*. A dispatched truck designs its own layout — the stack types, orientations, positions and layer compositions of its slots are variables of that truck and that period, and its load is assigned to those slots in the same period — so there is no catalogue of loading configurations. That is the decision Section 5 rests on: every row below carries either the triple (g, k, t) or none of it, so the model is block-angular over $G \times K \times T$ and the pricing problem of the Dantzig–Wolfe reformulation is obtained by deleting that quantifier and nothing else.

Throughout, $g \in G = \{s, p\}$ indexes nodes and directions (named by destination), $n \in N$ products, $r \in R$ RTI types, $h \in H$ stack types, $i, j \in I$ slot positions, $k \in K$ trucks and $t \in T$ periods, with $\rho(n)$ the RTI type product n travels in and $M(r) = \{n : \rho(n) = r\}$; variables whose direction is fixed by definition omit g. Table 3 is the nomenclature.

Table 3: Nomenclature. The pricing problem of Section 7 is one block of this model with the index (g, k, t) deleted, so it reuses these symbols with that index dropped; the three variables it adds are listed last.

Indices and sets	
G	Supplier and plant nodes, $G = \{s, p\}$; $g = s$ is the folded direction (plant to supplier), $g = p$ the full one.
N, R, H	Products, RTI types, stack types.
I	Stack-slot indices on a truck floor, bounded by (46).
K, T	Trucks available per direction and period; planning periods.
$\rho(n)$, $M(r)$	The RTI type product n travels in; the products type r carries, $M(r) = \{n : \rho(n) = r\}$.
$R(h)$	RTI types compatible with stack type h.
Parameters	
cK	Cost of dispatching one truck.
lt, SC	Lead time, $lt \geq 1$; coverage window, in periods.
i_{gn}, ie_{gr}	Initial inventory at g of full RTIs of n, of folded RTIs of r.
ic_{gn}	Inventory holding cost of n at g.
d_{gnt}	Production plan for n at g in t: filled at s, consumed at p.
a_{nt}, ae_{rt}	Pipeline arrivals at p, respectively s, in $t \leq lt$.
ss_{gnt}, sse_{gtr}	Coverage stock of full, respectively folded, RTIs (20).
w_n, w_r^{f}	Weight of a full RTI carrying n; of a folded RTI of type r. Both > 0.
pK^w, $W^{\max}$	Truck payload; per-stack crush-limit cap on the *total* weight of one stack, over all its layers.
pK, qK, rK	Truck floor length, floor width, inner height; $\Gamma_K = pK \cdot qK \cdot rK$.
α	Minimum truck utilisation, on whichever of volume or payload binds, $0 < \alpha \leq 1$.
p_h, q_h	Footprint length and width of stack type h.
r_{gr}, a_r, γ_{gr}	Height of one RTI of type r at node g (full at p, folded at s); its footprint area; its volume $\gamma_{gr} = a_r r_{gr}$.
n_{hr}	RTIs of type $r \in R(h)$ per layer of stack type h; one layer tiles the footprint, $n_{hr} a_r = p_h q_h$.
$\hat{w}_{gr}$	Weight per RTI in the layer-capacity form of the utilisation floor: $\hat{w}_{sr} = w_r^{\mathrm{f}}$, $\hat{w}_{pr} = \max_{n \in M(r)} w_n$.

η^K, η^I	Weights of the transport and inventory objectives; both 1 throughout.
M^x, M^y	Big-M constants of the non-overlap rows, $M^x = pK$, $M^y = qK$.
$\bar{u}^V_n$, $\bar{u}^{\mathrm{e}}_r$	$\lfloor W^{\max}/w_n \rfloor$ full RTIs of n per slot; $\lfloor pK^w/w^{\mathrm{f}}_r \rfloor$ folded RTIs of r per truck.
Decision variables — the truck and its layout	
Z_{gkt}	$\in \{0,1\}$. Truck k of direction g in t is configured, i.e. carries a layout.
Y_{gkt}	$\in \{0,1\}$. That truck is dispatched.
θ_{gkt}	$\in \{0,1\}$. Selects the resource on which its utilisation floor is required: volume if 1, payload if 0.
X_{gihkt}	$\in \{0,1\}$. Slot i holds a stack of type h.
L_{gihktr}	$\in \mathbb{Z}_{\geq 0}$. Layers of RTI type $r \in R(h)$ in slot i.
LX_{gihkt}, LY_{gihkt}	$\in \{0,1\}$. Stack h's length lies along the x-, respectively y-axis in slot i.
PX_{gikt}, PY_{gikt}	≥ 0. Bottom-left coordinates of slot i on the floor.
A, B, C, D (on $gijkt$)	$\in \{0,1\}$. Which of the four separating half-planes resolves the overlap of slots i and j.
Decision variables — load and inventory	
V_{kint}	$\in \mathbb{Z}_{\geq 0}$. Full RTIs of n in slot i of plant-bound truck k in t.
V^{e}_{krt}	$\in \mathbb{Z}_{\geq 0}$. Folded RTIs of type r on supplier-bound truck k in t.
I_{gnt}, Ie_{gtr}	$\in \mathbb{Z}_{\geq 0}$. End-of-period inventory of full RTIs of n, of folded RTIs of r, at g.
Derived quantities (abbreviations, (1)–(5))	
$\overline{V}_{nt}$, $\overline{V^{\mathrm{e}}}_{rt}$	Full RTIs of n delivered to the plant in t; folded RTIs of r returned to the supplier.
$\overline{V}_{knt}$	Full RTIs of n on plant-bound truck k in t.
$\widehat{X}_{gikt}$, $\overline{L}_{ghktr}$	Slot i is used; layers of type r under stack type h on that truck, over all slots.
Added by the reformulation (Sections 6–7)	
$\lambda^g_{\omega t}$	$\in \mathbb{Z}_{\geq 0}$. Trucks of class (g,t) dispatched carrying column ω.
$\pi_{nt}, \pi_{rt}, \mu_{gt}$	Duals of (55)–(56), respectively of (57) and of every further row a dispatch variable of (g,t) occupies.
n_{gt}, X	Dispatch total of class (g,t), $n_{gt} = \sum_\omega \lambda^g_{\omega t}$; the fleet $X = \sum_{g,t} n_{gt}$.

4.1 Derived quantities

Five sums recur, as *abbreviations* and not constraints.

$$\overline{V}_{nt} = \sum_{k\in K}\sum_{i\in I} V_{kint} \qquad \forall n \in N,\ t \in T \tag{1}$$

$$\overline{V^{\mathrm{e}}}_{rt} = \sum_{k\in K} V^{\mathrm{e}}_{krt} \qquad \forall r \in R,\ t \in T \tag{2}$$

$$\overline{V}_{knt} = \sum_{i\in I} V_{kint} \qquad \forall k \in K,\ n \in N,\ t \in T \tag{3}$$

$$\widehat{X}_{gikt} = \sum_{h\in H} X_{gihkt} \qquad \forall g \in G,\ i \in I,\ k \in K,\ t \in T \tag{4}$$

$$\overline{L}_{ghktr} = \sum_{i\in I} L_{gihktr} \qquad \forall g \in G,\ h \in H,\ k \in K,\ t \in T,\ r \in R(h) \tag{5}$$

The first two carry the method. The inventory block reads $\overline{V}_{nt}$ and $\overline{V^{\mathrm{e}}}_{rt}$ and nothing else about the trucks — not which truck carried what, nor how the load was placed.

4.2 Objective

Transport cost counts *dispatches*, so a configured but undispatched truck costs nothing; holding cost falls on erected RTIs.

$$\min\ z^K = \sum_{g\in G}\sum_{k\in K}\sum_{t\in T} cK \cdot Y_{gkt}, \qquad \min\ z^I = \sum_{g\in G}\sum_{n\in N}\sum_{t\in T} ic_{gn} \cdot I_{gnt} \tag{6}$$

$$\min\ \eta^K z^K + \eta^I z^I, \qquad \eta^K = \eta^I = 1 \text{ throughout}, \tag{7}$$

4.3 Inventory, flow balance and coverage

Arrivals are delayed; inventory itself accumulates one period at a time. For the plant,

$$I_{pnt} = I_{p,n,t-1} - d_{pnt} + \overline{V}_{n,t-lt} \qquad \forall n \in N,\ t \geq lt+1 \tag{8}$$

$$I_{pnt} = I_{p,n,t-1} - d_{pnt} + a_{nt} \qquad \forall n \in N,\ t \in \{2,\dots,lt\} \tag{9}$$

$$I_{pnt} = i_{pn} - d_{pnt} + a_{nt} \qquad \forall n \in N,\ t = 1 \tag{10}$$

$$Ie_{ptr} = Ie_{p,t-1,r} + \sum_{n \in M(r)} d_{pnt} - \overline{V^{\mathrm{e}}}_{rt} \qquad \forall r \in R,\ t \geq 2 \tag{11}$$

$$Ie_{ptr} = ie_{pr} + \sum_{n \in M(r)} d_{pnt} - \overline{V^{\mathrm{e}}}_{rt} \qquad \forall r \in R,\ t = 1 \tag{12}$$

Consuming a full RTI produces a folded one in the same period, which is why d_{pnt} drives (11). For the supplier,

$$I_{snt} = I_{s,n,t-1} + d_{snt} - \overline{V}_{nt} \qquad \forall n \in N,\ t \geq 2 \tag{13}$$

$$I_{snt} = i_{sn} + d_{snt} - \overline{V}_{nt} \qquad \forall n \in N,\ t = 1 \tag{14}$$

$$Ie_{str} = Ie_{s,t-1,r} + \overline{V^{\mathrm{e}}}_{r,t-lt} - \sum_{n \in M(r)} d_{snt} \qquad \forall r \in R,\ t \geq lt+1 \tag{15}$$

$$Ie_{str} = Ie_{s,t-1,r} + ae_{rt} - \sum_{n \in M(r)} d_{snt} \qquad \forall r \in R,\ t \in \{2,\dots,lt\} \tag{16}$$

$$Ie_{str} = ie_{sr} + ae_{rt} - \sum_{n \in M(r)} d_{snt} \qquad \forall r \in R,\ t = 1 \tag{17}$$

The lead time therefore acts on one side of a balance only — the sending node's stock falls in the dispatch period, the receiving node's rises lt periods later — an alignment Section 8 uses twice. Coverage stock is enforced at both nodes and on both kinds of stock,

$$I_{gnt} \ \geq \ ss_{gnt} \qquad \forall g \in G,\ n \in N,\ t \in T \tag{18}$$

$$Ie_{gtr} \ \geq \ sse_{gtr} \qquad \forall g \in G,\ r \in R,\ t \in T \tag{19}$$

the requirement being that node's own series over the next SC periods:

$$ss_{gnt} = \sum_{j=1}^{SC} d_{g,n,\tau(t,j)}, \qquad sse_{gtr} = \sum_{n \in M(r)} ss_{gnt}, \qquad \tau(t,j) = \begin{cases} t+j, & t+j \leq |T|, \\ |T| - SC + j, & \text{otherwise.} \end{cases} \tag{20}$$

Near the end of the horizon (20) wraps the window back onto the last SC periods rather than truncating it, following (Díaz-Madroñero, Mula, and Peidro 2014): truncating lowers the requirement of the final periods and with it the optimal value, so the tail rule belongs to the problem statement.

4.4 The layout of a truck

Everything from here to (43) is written for one truck: one direction g, one index k, one period t. At most one stack type occupies a slot and only a configured truck has slots; a stack is oriented along one axis or the other; its height cannot exceed the trailer's, which with $r_{gr} > 0$ also empties an unused slot:

$$\widehat{X}_{gikt} \ \leq \ Z_{gkt} \qquad \forall i \in I \tag{21}$$

$$LX_{gihkt} + LY_{gihkt} \ = \ X_{gihkt} \qquad \forall h \in H,\ i \in I \tag{22}$$

$$\sum_{r \in R(h)} r_{gr}\, L_{gihktr} \ \leq \ rK \cdot X_{gihkt} \qquad \forall h \in H,\ i \in I \tag{23}$$

Slots must not overlap — for each pair $i < j$, the usual four-way disjunction closed by a cover row — and every slot lies inside the floor:

$$PX_{gikt} + \sum_{h \in H} \big(p_h LX_{gihkt} + q_h LY_{gihkt}\big) \le PX_{gjkt} + M^x(1 - A_{gijkt}) \tag{24}$$

$$PX_{gjkt} + \sum_{h \in H} \big(p_h LX_{gjhkt} + q_h LY_{gjhkt}\big) \le PX_{gikt} + M^x(1 - B_{gijkt}) \tag{25}$$

$$PY_{gikt} + \sum_{h \in H} \big(q_h LX_{gihkt} + p_h LY_{gihkt}\big) \le PY_{gjkt} + M^y(1 - C_{gijkt}) \tag{26}$$

$$PY_{gjkt} + \sum_{h \in H} \big(q_h LX_{gjhkt} + p_h LY_{gjhkt}\big) \le PY_{gikt} + M^y(1 - D_{gijkt}) \tag{27}$$

$$A_{gijkt} + B_{gijkt} + C_{gijkt} + D_{gijkt} \;\ge\; \widehat{X}_{gikt} + \widehat{X}_{gjkt} - 1 \tag{28}$$

$$PX_{gikt} + \sum_{h \in H} \big(p_h LX_{gihkt} + q_h LY_{gihkt}\big) \le pK \tag{29}$$

$$PY_{gikt} + \sum_{h \in H} \big(q_h LX_{gihkt} + p_h LY_{gihkt}\big) \le qK \tag{30}$$

The containment rows are unconditional, which is what lets the non-overlap rows take the tightened $M^x = pK$, $M^y = qK$.

4.5 Full truckload on every dispatched truck

A dispatched truck must be full, and it can be full in two ways: it may run out of space or of payload. A volume-only floor is not a conservative simplification but an infeasible one — filling the fraction $\alpha = 0.55$ of a 97.92 m^3 trailer ($13\,600 \times 2\,400 \times 3\,000$ mm) with freight at 579 $\mathrm{kg/m}^3$ weighs 31.2 t against a 24 t cap, and seven of the industrial catalogue's fifteen types are denser still. Summing the two ratios is the opposite error: it accepts a truck 35% full by volume and 20% by weight. We therefore require a fraction α of *either* resource, a binary θ_{gkt} selecting which one binds, on the load the truck carries with each commodity's own weight:

$$\sum_{n \in N} \gamma_{p\rho(n)}\, \overline{V}_{knt} \;\ge\; \alpha\, \Gamma_K \big(Y_{pkt} - 1 + \theta_{pkt}\big) \qquad \forall k \in K,\ t \in T \tag{31}$$

$$\sum_{n \in N} w_n\, \overline{V}_{knt} \;\ge\; \alpha\, pK^w \big(Y_{pkt} - \theta_{pkt}\big) \qquad \forall k \in K,\ t \in T \tag{32}$$

$$\sum_{r \in R} \gamma_{sr}\, V^{\mathrm{e}}_{krt} \;\ge\; \alpha\, \Gamma_K \big(Y_{skt} - 1 + \theta_{skt}\big) \qquad \forall k \in K,\ t \in T \tag{33}$$

$$\sum_{r \in R} w^{\mathrm{f}}_r\, V^{\mathrm{e}}_{krt} \;\ge\; \alpha\, pK^w \big(Y_{skt} - \theta_{skt}\big) \qquad \forall k \in K,\ t \in T \tag{34}$$

Exactly one of each pair is active at $Y_{gkt} = 1$, so each is the disjunction "full by volume *or* full by payload". The same disjunction on the layer *capacity* the stacks provide rather than on the load they receive,

$$\sum_{h \in H} \sum_{r \in R(h)} \gamma_{gr}\, n_{hr}\, \overline{L}_{ghktr} \;\ge\; \alpha\, \Gamma_K \big(Y_{gkt} - 1 + \theta_{gkt}\big), \quad \sum_{h \in H} \sum_{r \in R(h)} \hat{w}_{gr}\, n_{hr}\, \overline{L}_{ghktr} \;\ge\; \alpha\, pK^w \big(Y_{gkt} - \theta_{gkt}\big) \tag{35}$$

is a valid tightening, kept because it lets a solver establish a layout before choosing a product assignment. Which form is the *requirement* matters. (31)–(34) is, full truckload being a property of the freight that moves; (35) follows from it because a slot never receives more RTIs of type r than its layers hold and $\hat{w}_{gr} \ge w_n$ for every $n \in M(r)$, and taken *alone* would admit trucks that are not full. That inequality is also why $\hat{w}_{pr} = \max_{n \in M(r)} w_n$ is the family *maximum*: the family *minimum* would assert that *every* product assignment of this layout fills the truck, cutting off feasible loads.

4.6 Dispatch, loading and weight

A dispatched truck must be configured,

$$Y_{gkt} \;\le\; Z_{gkt} \qquad \forall g \in G,\ k \in K,\ t \in T, \tag{36}$$

while the converse fails: $Z_{gkt} = 1$ with $Y_{gkt} = 0$ is a truck carrying a layout and no freight, which in the reformulated block is exactly the empty column (Section 7).

Full RTIs in a slot cannot exceed the layer capacity of the stack placed there, products sharing an RTI type partitioning it; folded RTIs are capped by the truck's aggregate capacity, folded weight being deterministic in the type. Both loads are gated on the dispatch, at their own analytic bounds rather than a generic big-M:

$$\sum_{n \in M(r)} V_{kint} \;\le\; \sum_{h\,:\,r \in R(h)} n_{hr}\, L_{p,i,h,k,t,r} \qquad \forall k \in K,\ i \in I,\ r \in R,\ t \in T \tag{37}$$

$$V^{\mathrm{e}}_{krt} \;\le\; \sum_{h\,:\,r \in R(h)} n_{hr}\, \overline{L}_{s,h,k,t,r} \qquad \forall k \in K,\ r \in R,\ t \in T \tag{38}$$

$$V_{kint} \le \bar{u}^{V}_{n}\, Y_{pkt} \qquad \forall k \in K,\ i \in I,\ n \in N,\ t \in T \tag{39}$$

$$V^{\mathrm{e}}_{krt} \le \bar{u}^{\mathrm{e}}_{r}\, Y_{skt} \qquad \forall k \in K,\ r \in R,\ t \in T \tag{40}$$

The payload is a property of the truck, the crush limit of each stack; on plant-bound trucks that cap is *period-exact*, evaluated on the mix actually loaded in the slot, on supplier-bound trucks deterministic in the layout:

$$\sum_{n \in N} w_n\, \overline{V}_{knt} \;\le\; pK^{w}\, Y_{pkt}, \qquad \sum_{r \in R} w^{\mathrm{f}}_{r}\, V^{\mathrm{e}}_{krt} \;\le\; pK^{w}\, Y_{skt} \qquad \forall k \in K,\ t \in T \tag{41}$$

$$\sum_{n \in N} w_n\, V_{kint} \;\le\; W^{\max} \qquad \forall k \in K,\ i \in I,\ t \in T \tag{42}$$

$$\sum_{h \in H} \sum_{r \in R(h)} w^{\mathrm{f}}_{r}\, n_{hr}\, L_{s,i,h,k,t,r} \;\le\; W^{\max} \qquad \forall i \in I,\ k \in K,\ t \in T \tag{43}$$

where (41) names both payload rows and (39) is tighter in the relaxation than their implication, since what bounds a slot's content is the crush cap.

4.7 Symmetry, slot-set sizing and data assumptions

The slots of a truck are interchangeable, and so are the trucks of a direction and period. Both are ordered:

$$\widehat{X}_{g,i+1,k,t} \;\le\; \widehat{X}_{gikt} \qquad \forall g \in G,\ k \in K,\ t \in T,\ i < |I| \tag{44}$$

$$Y_{g,k+1,t} \;\le\; Y_{gkt} \qquad \forall g \in G,\ t \in T,\ k < |K| \tag{45}$$

Re-indexing makes (45) valid, but it removes only *which* subset of trucks is used: loads and layouts stay interchangeable among those used, so a plan dispatching m trucks in a period still appears $m!$ times — the difficulty Section 5 removes by aggregation rather than by a row. The slot count is a modelling parameter, and

$$|I| \;\le\; \Big\lfloor (pK \cdot qK) \big/ \min_{h \in H} p_h q_h \Big\rfloor \tag{46}$$

eliminates provably unused slot variables without cutting any feasible layout. It is more than preprocessing: a placement model given fewer slots than a layout needs can report infeasible while the load is placeable (Section 7).

Four properties of the data are assumed: ρ is a partition; $lt \ge 1$, which the lagged terms require; all weights and heights are strictly positive, which makes $\bar{u}^{V}_{n}$ and $\bar{u}^{\mathrm{e}}_{r}$ well defined and lets (23) empty an unused slot; and the footprint geometry is consistent, one layer of stack type h tiling its footprint, $n_{hr} a_r = p_h q_h$, with the volume charged to an RTI its bounding box, $\gamma_{gr} = a_r r_{gr}$. *The last is a departure*, and Proposition 2 and the floor-area inequality (72) are both false without it. It costs nothing here, both instance families building one stack type per distinct RTI footprint, so $n_{hr} = 1$ and the relation is an identity.

4.8 Domains

$$Z_{gkt},\, Y_{gkt},\, \theta_{gkt},\, X_{gihkt},\, LX_{gihkt},\, LY_{gihkt},\, A_{gijkt},\, B_{gijkt},\, C_{gijkt},\, D_{gijkt} \in \{0,1\},$$
$$L_{gihktr},\, V_{kint},\, V^{\mathrm{e}}_{krt},\, I_{gnt},\, Ie_{gtr} \in \mathbb{Z}_{\ge 0}, \qquad PX_{gikt},\, PY_{gikt} \ge 0.$$

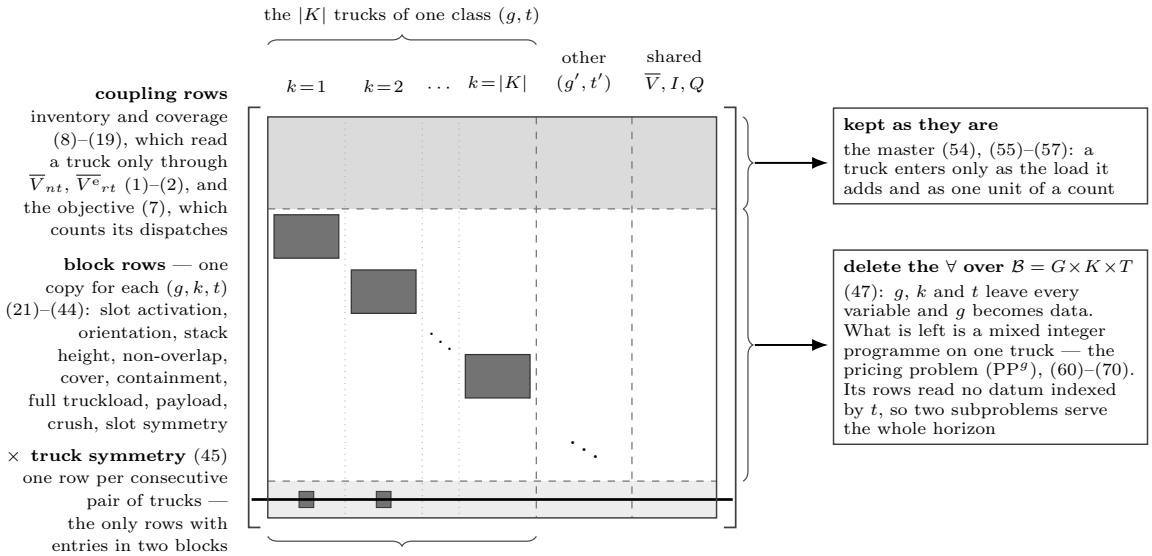


Figure 2: The decomposition is read off the formulation, not chosen by analogy. Every geometric, stacking, crush, payload and full-truckload row of Section 4 is confined to one truck, so the matrix is block-angular over $\mathcal{B} = G \times K \times T$; deleting the quantifier over $\mathcal{B}$ from those rows leaves a mixed integer programme on a single truck, and that programme is the pricing problem. Nothing is invented for the subproblem and no geometry stays behind in the master, which is why a branching row added to the master never changes what the subproblem is. Table 4 is the same statement row by row.

5 Why branch and price, and why this decomposition

Two structural difficulties stand between the formulation of Section 4 and an industrial instance. The model carries a complete two-dimensional packing problem for every truck of every direction and period — $2|K||T|$ replicas, each contributing $4\binom{|I|}{2}$ big-M disjunctions (24)–(27) whose linear relaxation is weak — and those replicas are interchangeable, any two used trucks of one direction and period being able to swap layouts and loads, so every solution appears a factorial number of times. The second difficulty scales worst and does not depend on the loading geometry at all, being present in the volumetric benchmark of Appendix A exactly as in 3D-SCPTOP.

The Dantzig–Wolfe reformulation below (Dantzig and Wolfe 1960; Barnhart et al. 1998; Desaulniers, Desrosiers, and Solomon 2005) removes both, and removes the first without restricting anything. Figure 2 is the picture, Table 4 the row-by-row detail.

Blocks. Take the block index to be $\mathcal{B} = G \times K \times T$. Every row from (21) to (45) — the first column of Table 4 names them — carries a single direction, truck and period as a fixed $\forall$ index or a literal, and none couples two values of g, k or t; the exception is (45), relating truck $k+1$ to truck k. The remaining rows are the objective, the inventory and coverage block (8)–(19) and the two definitions (1)–(2), and they see the blocks *only* through those aggregates and the count of dispatches z^K charges for. Block-angularity over $\mathcal{B}$ is therefore a property of the formulation, not a discovery about it.

Delete the quantifier. Fix a class (g,t), take the block rows, and *remove the* $\forall$ *over* $\mathcal{B}$: drop the indices g, k and t from every variable and read g as data. The stack-height row, for instance, goes

$$\underbrace{\sum_{r\in R(h)} r_{gr} L_{gihktr} \le rK\, X_{gihkt} \quad \forall g,h,i,k,t}_{\text{3D-SCPTOP, (23)}} \longmapsto \underbrace{\sum_{r\in R(h)} r_{gr} L_{ihr} \le rK\, X_{ih} \quad \forall h,i,}_{\text{(PP}^g\text{), (63)}} \tag{47}$$

and every other block row the same way; what is left is a mixed integer programme over one truck, the (PP^g) of Section 7. Nothing is invented for the subproblem and nothing of the geometry is left behind in the master. Two rows do not make the journey: (45), which the aggregation disposes of, and (36), whose two binaries collapse into one (Section 7).

Identical blocks. K is anonymous, so the blocks of a fixed (g, t) are identical and are aggregated: in place of one copy per truck, the reformulation carries one non-negative integer variable per block *solution*, counting how many of the class's blocks take it, and one row per class bounding that count by $|K|$. This is where the second difficulty disappears — the trucks are never distinguished, so there is no permutation to search — and it is why the dropped row costs nothing: counts carry no order, so the master has no variable in which to express (45). The converse direction of Proposition 1 assigns trucks *in index order*, so the equivalence holds against the compact model *with* that row in force.

Classes. The classes are the pairs (g, t), and within a class the block rows are pinned to the class's g, so there is one pricing problem per *direction*. The pricing rows read no datum indexed by t — the trailer, the stack types, the crush limit and the floor are time-invariant — so the periods of one direction differ *only in their duals*, one column set serves them all, and the whole horizon costs two pricing problems rather than $2|T|$.

Discretisation. Every block variable that reaches the master is integer, so a block solution may be taken to be an *integer* point of its pricing problem completed by placement coordinates. The multipliers are then integer rather than merely non-negative, and a fractional one is a genuine integrality violation.

Table 4: The block rows of Section 4 and the rows of (PP^g) they become when the $\forall$ over $\mathcal{B} = G \times K \times T$ is deleted. The two directions share the geometry, so it is written once with g as data; the direction-specific rows are those of Section 7.1. One row of (PP^g) is not a transcription and is marked as such.

Row	Section 4	(PP^g)
Block rows: the pricing problem		
Slot active	(21)	(61)
Orientation	(22)	(62)
Stack height	(23)	(63)
Non-overlap	(24)–(27)	(64)–(65)
Cover	(28)	(66)
Containment	(29)–(30)	(67)
Full truckload, volume	(31), (33)	(69)
Full truckload, payload	(32), (34)	(70)
The same on capacity	(35)	(71), re-gated on Z
Occupied floor area	— (no counterpart)	(72), a valid inequality
Slot capacity	(37)	(73)
Folded capacity	(38)	(77)
Dispatch gate, full	(39)	(74)
Dispatch gate, folded	(40)	(78)
Payload, both directions	(41)	(75), (79)
Crush limit, plant-bound	(42)	(76)
Crush limit, supplier-bound	(43)	(80)
Slot symmetry	(44)	(68)
Transport cost	(6)	$\eta^K cK \cdot Z$ in (60)
Dispatch	(36)	deleted; Z carries the truck
Truck symmetry	(45)	dropped in the aggregation
Coupling rows: the master		
Aggregate flows	(1)–(2)	(55)–(56)
Fleet per class	$\|K\|$ trucks per (g, t)	(57)
Inventory and coverage	(8)–(19)	verbatim in the master
Holding cost	(6)	second term of (54)

6 The master problem

6.1 Columns

A column of a direction is the image of a feasible point of that direction's pricing problem, together with the point itself as a witness; Ω^p and Ω^s are the two column sets, indexed by ω. A *plant-bound* column $\omega \in \Omega^p$ is a vector $b_\omega = (b_{\omega n})_{n \in N}$ of full RTIs — the value of $\overline{V}_n$ at the point — for which there exist a slot count $m \le |I|$, stack types $h_1, \dots, h_m \in H$, orientations, bottom-left coordinates (x_i, y_i), layer counts ℓ_{ir} and per-slot product loads v_{in} such that

$$\text{the } m \text{ footprints, rotated as chosen, do not overlap and lie inside } pK \times qK, \tag{48}$$

$$\textstyle\sum_{r \in R(h_i)} r_{pr} \ell_{ir} \;\le\; rK \qquad \forall i \le m, \tag{49}$$

$$\textstyle\sum_{n \in M(r)} v_{in} \;\le\; n_{h_i r} \ell_{ir} \qquad \forall i \le m,\ r \in R, \tag{50}$$

$$\textstyle\sum_{n \in N} w_n v_{in} \;\le\; W^{\max} \qquad \forall i \le m, \tag{51}$$

$$\textstyle\sum_{n \in N} w_n b_{\omega n} \;\le\; pK^w, \qquad \max\Big\{ \frac{\sum_n \gamma_{p\rho(n)} b_{\omega n}}{\Gamma_K},\ \frac{\sum_n w_n b_{\omega n}}{pK^w} \Big\} \;\ge\; \alpha, \tag{52}$$

$$b_{\omega n} = \textstyle\sum_{i \le m} v_{in} \qquad \forall n \in N. \tag{53}$$

These are the block rows of Section 4 read on one truck (Table 4); in (50) $n_{h_i r} \ell_{ir}$ is read as 0 for $r \notin R(h_i)$, and it is an *inequality* because a partly filled layer is a legal compact layout.

A *supplier-bound* column is the analogous object $e_\omega = (e_{\omega r})_{r \in R}$ over folded heights, weights and volumes, with (43) as its per-slot crush condition and $e_{\omega r} \le \sum_{i \le m} n_{h_i r} \ell_{ir}$, which is (38), in place of (50) and (53); the weight of a folded RTI depends only on its type, so no product assignment is needed. We write (48)–(53) from here on for either direction, writing $\text{image}_c(\omega)$ for the entry of b_ω or e_ω at the *commodity* c — a product when $g = \text{p}$, an RTI type when $g = \text{s}$. Every column costs $c_\omega = \eta^K cK$, and Ω^g holds the *non-empty* columns only, the all-zero point being the slack of (57).

Two features of this definition carry the method: the column holds the per-slot assignment v_{in}, so (51) is the period-exact cap (42) and not the family surrogate a catalogue of capacity-only configurations must use, and it retains its witnessing placement, so a plan is a set of drawings.

6.2 The master

Let $\lambda^g_{\omega t} \in \mathbb{Z}_{\ge 0}$ be the number of trucks of class (g, t) dispatched carrying column ω. The master problem (MP) is

$$\min \;\; \sum_{g \in G} \sum_{t \in T} \sum_{\omega \in \Omega^g} c_\omega \, \lambda^g_{\omega t} \;+\; \eta^I \sum_{g \in G} \sum_{n \in N} \sum_{t \in T} i c_{gn} I_{gnt} \tag{54}$$

subject to

$$\overline{V}_{nt} = \sum_{\omega \in \Omega^p} b_{\omega n} \, \lambda^p_{\omega t} \qquad \forall n \in N,\ t \in T \tag{55}$$

$$\overline{V^{\text{e}}}_{rt} = \sum_{\omega \in \Omega^s} e_{\omega r} \, \lambda^s_{\omega t} \qquad \forall r \in R,\ t \in T \tag{56}$$

$$\sum_{\omega \in \Omega^g} \lambda^g_{\omega t} \le |K| \qquad \forall g \in G,\ t \in T \tag{57}$$

$$(8)\text{–}(19) \qquad \text{verbatim}$$

$$\lambda^g_{\omega t} \in \mathbb{Z}_{\ge 0}, \quad I_{gnt},\, Ie_{gtr} \in \mathbb{Z}_{\ge 0}, \quad \overline{V}_{nt},\, \overline{V^{\text{e}}}_{rt} \in \mathbb{Z}_{\ge 0} \text{ (implied by (55)–(56))}.$$

That is the whole master: every column costs the same, so the transport term is cK times the number dispatched, and (55)–(56) are (1)–(2) with each truck's load replaced by its column's image. Since Ω^g holds the non-empty columns only, $\overline{\lambda}_{gt} = \sum_\omega \lambda^g_{\omega t}$ is the number *dispatched* by class (g, t), which is what Section 8 branches on.

Proposition 1 (equivalence)**.** *MP and 3D-SCPTOP have the same optimal value.*

Proof. Each used truck (g, k, t) of a 3D-SCPTOP solution induces a load vector satisfying its direction's column conditions, read off its own layout with $\ell_{ir} = L_{g,i,h_i,k,t,r}$ and, at the plant, $v_{in} = V_{kint}$; every one of those conditions is a row the truck already satisfies. Counting trucks by load vector gives a feasible MP

solution of equal cost, since transport cost depends only on the number of used trucks and the inventory rows see only the aggregates (55)–(56) reproduce. Conversely, give each truck of an MP solution the layout witnessing its column: the blocks are identical, so any assignment serves, and in index order the result also satisfies (45). □

No hypothesis is needed because there is no catalogue of layouts to be sized: in Section 4 a truck's layout is its own. MP is therefore the same problem written differently, and any bound derived from it bounds the original.

6.3 The restricted master and the reduced cost

$|\Omega^g|$ is astronomical, so MP is solved over a subset $\bar{\Omega}^g$ with integrality relaxed — the restricted master problem (RMP) — whose duals the pricing problems use to look for columns of negative reduced cost (Figure 3). With π_{nt}, π_{rt} the duals of (55)–(56) and $\mu_{gt} \leq 0$ that of (57), the reduced cost of a plant-bound column dispatched in t is

$$\bar{c}(\omega, t) \;=\; c_\omega - \sum_{n \in N} \pi_{nt}\, b_{\omega n} - \mu_{pt}, \tag{58}$$

and analogously at the supplier. The pricing problem of class (g, t) minimises $c(x) - \sum_c \pi_{ct}\, \mathrm{image}_c(x)$ over the feasible points of the direction's block, so with optimal value $z^\star_{gt}$ it gives $\min_\omega \bar{c}(\omega, t) = z^\star_{gt} - \mu_{gt}$, and a column enters when that is negative.

each column comes from the pricing problem (Figure 4): one truck load, with its layout →



	$\lambda^p_{\omega t}$ full RTIs	$\lambda^s_{\omega' t}$ folded RTIs	$\overline{V}_{nt}$, $\overline{V^{\mathrm{e}}}_{rt}$ aggregate	I_{gnt}, Ie_{gtr} stock	artificial columns† priced M	to the pricing problem, rounded up
objective (54)	cK	cK	·	ic_{gn}	M	
aggregate flow (55) one row per product n, period t	$b_{\omega n}$	·	-1	·	$+1$	π_{nt}
aggregate flow (56) one row per RTI type r, period t	·	$e_{\omega' r}$	-1	·	$+1$	π_{rt}
truck count (57) one row per direction and period	1	1	·	·	·	μ_{gt}
cumulative cover cut (59)† one row per direction and period, over the dispatches with $\tau \leq t - lt$	1	1	·	·	$+1$	folds into μ_{gt}
inventory balance (8)–(17) the receiver's arrival lagged by lt	·	·	± 1	± 1	·	no λ: pricing feels these rows only through π
coverage stock (18)–(19)	·	·	·	$+1$	$+1$	
rows a catalogue of loading configurations would add	**none.** No slot, stack type, orientation, coordinate or layer count is a variable of the master: the geometry stays inside the column, with its witnessing layout (Figure 4).					

† Rows and columns of the *restricted* master; (54)–(57) together with (8)–(19) are the master itself.

Figure 3: The master is a flow model that has never seen a box. A column is one truck load, and the only rows it enters are the two aggregate-flow definitions, one truck count per direction and period, and the cumulative cover cut — every one of which merely counts or sums what the load contains. No slot, stack type, orientation, coordinate or layer count is a variable here, so there is no catalogue of loading configurations to enumerate, and no row written over these variables can reach the geometry; the inventory and coverage rows see the trucks only through $\overline{V}$ and $\overline{V^{\mathrm{e}}}$, and all that ever reaches the pricing problem is π and μ. The non-negative artificials keep the restricted master feasible, and its duals defined, for any pool — a solution that uses them is not a plan.

The prices, irrational in general, must be rounded *upwards* before entering an integer-coefficient subproblem, never to nearest, so that the rounded objective dominates the true one and the subproblem's value stays an over-estimate of the true maximum, as the certificate below requires. Rounding to nearest invalidates that certificate and, with reduced costs near zero, returns a value just above the threshold so that convergence is never declared.

The RMP is *elastic*: non-negative artificials sit on (55)–(56), on the coverage rows and on the cuts below at a large price, so it is feasible for any pool and always yields duals, and a solution with positive artificials is not a plan. The elasticity is one-sided where it matters: a definition row reads $\overline{V} = \sum_\omega b\lambda + \text{art}$, $\text{art} \geq 0$, so slack can only enlarge a delivery — which is why stock non-negativity stays rigid against a fleet lower bound.

It also carries cumulative cover cuts. With ρ^g_{ct} the cumulative requirement of commodity c at the receiving node g through t — demand to date plus the coverage stock of t, net of initial stock and freight in transit, floored at zero — and $\bar{u}^g = \min\{\lfloor pK^w / \min_c w_c \rfloor, \lfloor \Gamma_K / \min_c \gamma_{gc} \rfloor\}$ an upper bound on the units one truck holds, every $t > lt$ gives

$$\sum_{\tau \leq t-lt} \sum_{\omega \in \Omega^g} \lambda^g_{\omega\tau} \;\geq\; \max\Big\{ \Big\lceil \tfrac{\sum_c \gamma_{gc}\rho^g_{ct}}{\Gamma_K} \Big\rceil,\; \Big\lceil \tfrac{\sum_c w_c \rho^g_{ct}}{pK^w} \Big\rceil,\; \Big\lceil \tfrac{\sum_c \rho^g_{ct}}{\bar{u}^g} \Big\rceil \Big\}. \tag{59}$$

Each ratio divides a requirement by the most one truck can carry of that resource, so over-estimating $\bar{u}^g$ only weakens the cut while the shift by lt is mandatory. Its duals fold into μ_{gt} and leave (58) exact, which is possible exactly when a row $\sum_\omega \phi(\omega)\lambda_\omega \lessgtr \beta$ has ϕ constant within a class or affine in the column's image. Such a row is *robust*: it changes the duals and nothing else. Every row this paper adds is of that kind, which is why no branching decision reaches a pricing problem.

Finally the RMP is *period-scoped*: a column priced at t receives a dispatch variable at t alone. That changes the restricted master's size and not the master's, so a sweep that finds nothing anywhere still proves the restricted LP equal to the master LP.

6.4 What may be reported as a lower bound

The natural candidate is not a bound at all. The RMP is a *restriction* of MP, so $z_{\text{RMP}} \geq z_{\text{MP}}$: until column generation converges the RMP value is an *upper* bound on the master LP. Its one legitimate use is as a filter.

A certificate needs a full sweep. When the pricing problem has *proved* that

$$\zeta_{gt} \;:=\; \max_{\omega \in \Omega^g} \sum_c \pi_{ct}\, \text{image}_c(\omega) \;\leq\; \eta^K cK - \mu_{gt} \qquad \forall g \in G,\; t \in T,$$

nothing outside $\bar{\Omega}^g$ prices out, the duals are feasible for the dual of the full MP, and weak duality gives $z_{\text{RMP}} = z_{\text{MP}} \leq z^\star_{\text{3D-SCPTOP}}$: a bound as good as the sweep behind it (Algorithm 1). Memoising duals, and pricing only the pairs whose aggregates are fractional, are legitimate while columns are still arriving and forfeit the right to certify; so does one pair stopped on its time limit. One case looks like a time limit and is not: a call stopped by a cutoff *without* returning a point ran out of search tree rather than out of time and does prove that no column beats the threshold, the two being separated only by whether a point came back. Every branching fix must be released before the sweep, a restricted LP sitting above the region's own. The bound is then the *raw* LP value, penalty included; positive artificials at convergence mean the region admits no plan at all and it is pruned as infeasible.

A bound without convergence. A complete sweep that finds columns still yields a Lagrangian bound, $\bar{z} + \sum_{g,t} |K| \min(0, \bar{c}^\star_{gt})$ with the count row (57) as multiplier, provided each $\bar{c}^\star_{gt}$ bounds the pricing *maximum* from above — for which dropping the geometry, the integrality and the floor leaves a two-resource knapsack. It is too weak to rely on here: a reduced cost of $-\varepsilon$ at every pair subtracts $2|T||K|\varepsilon$ from $\bar{z}$, where the distance to make up is a fraction of one truck.

A bound by inclusion. Every 3D lower bound in this paper is an instance of the following, which is affordable where the certificate above — with the geometric pricing problem as oracle — is not.

Proposition 2 (bound by inclusion)**.** *Let $\Omega^g_{\text{vol}} \supseteq \Omega^g$ be the loads that fit the trailer's volume and the payload and clear the full-truckload floor on the binding resource, and z_{vol} the certified LP value of the master* (54)–(57) *over them. Then $z_{\text{vol}} \leq z^\star_{\text{3D-SCPTOP}}$, and the same holds inside any region defined by bounds on the fleet vector.*

Algorithm 1 CERTIFY($\bar{\Omega}, \mathcal{R}$) — the sweep a lower bound rests on. No memoisation, no restriction to the pairs a branch has touched, every branching fix released.

release every branching fix; impose $\mathcal{R}$ on the RMP as fleet rows
loop
  solve the LP over $\bar{\Omega}$ inside $\mathcal{R}$
  if infeasible **then**
    return $(\infty,$ true) ▷ region refuted
  end if
  proved $\leftarrow$ true
  for all $(g,t) \in G \times T$ **do**
    $\zeta_{gt} \leftarrow$ PRICE(π, μ, g, t) to proven optimality, duals rounded *up*
    if the call returned neither OPTIMAL nor INFEASIBLE, and was not an objective-limit stop with *no* point returned **then**
      proved $\leftarrow$ false ▷ a cutoff that returns nothing is still a proof
    end if
  end for
  if some ζ_{gt} prices out **then**
    add its columns to $\bar{\Omega}$ and **continue**
  end if
  if artificials > 0 **then**
    return $(\infty,$ true) ▷ no plan over Ω^g lies in $\mathcal{R}$
  end if
  return (raw LP value, proved) ▷ a bound only if proved
end loop

Proof. The inclusion rests on two conventions of Section 4: $\Gamma_K = pK \cdot qK \cdot rK$, and the volume γ_{gr} charged to an RTI is its bounding box, with no allowance for void space inside it or clearance between stacks, so that by footprint consistency one layer of stack type h occupies exactly $n_{hr}\gamma_{gr} = p_h q_h r_{gr}$. A placement satisfying (48)–(53) puts its RTIs in disjoint boxes inside the trailer — footprints non-overlapping and inside $pK \times qK$, each stack under rK by (49) — so their bounding-box volumes sum to at most Γ_K, its weight is at most pK^w, and (52) is the same floor in both sets; hence $\Omega^g \subseteq \Omega^g_{\text{vol}}$. Only the column set depends on the load model, so the 3D master is the volumetric master with columns removed: its LP value is at least z_{vol}, and its optimum at least its LP value. □

The consequence is a division of labour. A geometric sweep costs $2|T|$ calls at the pricing limit; over the volumetric relaxation the same sweep is a two-constraint knapsack oracle and closes in seconds. The method therefore asks geometric pricing only for columns and takes its bound from the volumetric relaxation, so the gap between them is a genuine optimality gap for the 3D problem.

7 The pricing problem

By the Blocks and Classes steps of Section 5 the pricing problem of class (g,t) *is* the block of 3D-SCPTOP with the quantifier over $\mathcal{B}$ removed: the same rows, with g supplying the data and the master the duals, the only rows below with no counterpart in Section 4 being the valid inequalities (72). There is one such problem per direction, written (PP^g). With c the commodity index of Section 6.1, image$_c$, w_c, γ_{gc} and r_{gc} are $\overline{V}_n, w_n, \gamma_{p\rho(n)}, r_{p\rho(n)}$ respectively $V^{\text{e}}_r, w^{\text{f}}_r, \gamma_{sr}, r_{sr}$; the partition programme (82)–(86) uses the same convention. Figure 4 draws what the block designs.

$$\min\ \eta^K cK\cdot Z\ -\ \sum_c \pi_{ct}\,\mathrm{image}_c \qquad \bar{c}=z-\mu_{gt} \tag{60}$$

$$\widehat{X}_i \le Z \qquad \forall i\in\mathcal{I} \tag{61}$$

$$LX_{ih}+LY_{ih}=X_{ih} \qquad \forall h\in H,\ i\in\mathcal{I} \tag{62}$$

$$\sum_{r\in R(h)} r_{gr}\,L_{ihr}\le rK\cdot X_{ih} \qquad \forall h\in H,\ i\in\mathcal{I} \tag{63}$$

$$PX_i+\delta^x_i\le PX_j+M^x(1-A_{ij}),\quad PX_j+\delta^x_j\le PX_i+M^x(1-B_{ij}) \qquad \forall i<j \tag{64}$$

$$PY_i+\delta^y_i\le PY_j+M^y(1-C_{ij}),\quad PY_j+\delta^y_j\le PY_i+M^y(1-D_{ij}) \qquad \forall i<j \tag{65}$$

$$A_{ij}+B_{ij}+C_{ij}+D_{ij}\ge \widehat{X}_i+\widehat{X}_j-1 \qquad \forall i<j \tag{66}$$

$$PX_i+\delta^x_i\le pK,\qquad PY_i+\delta^y_i\le qK \qquad \forall i\in\mathcal{I} \tag{67}$$

$$\widehat{X}_{i+1}\le\widehat{X}_i \qquad \forall i\le|\mathcal{I}|-1 \tag{68}$$

$$\sum_c \gamma_{gc}\,\mathrm{image}_c\ \ge\ \alpha\,\Gamma_K\,(Z-1+\theta) \tag{69}$$

$$\sum_c w_c\,\mathrm{image}_c\ \ge\ \alpha\,pK^w\,(Z-\theta) \tag{70}$$

with $\delta^x_i=\sum_h(p_hLX_{ih}+q_hLY_{ih})$ and $\delta^y_i=\sum_h(q_hLX_{ih}+p_hLY_{ih})$ the footprint of slot i in its chosen orientation, and (67) naming both containment rows and (64), (65) the four non-overlap rows, so that at $Z=1$ the pair (69)–(70) is the disjunction (52).

Deleting a variable is a claim about the feasible set. With no loading-configuration index, Z_{gkt} and Y_{gkt} differ at exactly one kind of point, $Z=1$ with $Y=0$: a truck carrying a layout and no freight, whose image and cost are both 0 by (39)–(40) and (6) — precisely those of the empty column. Identifying the two binaries therefore removes no element of Ω^g, and the master, which sees only images and costs, cannot tell the difference. The block keeps Z, charges it $\eta^K cK\cdot Z$ and hangs every load gate on it; the empty column, analytically known with reduced cost $-\mu_{gt}\ge 0$, is excluded by fixing $Z=1$.

Four implied rows are available and the oracles behind Section 9 carry the second pair — two restating the floor on the layer *capacity* rather than on the load, and two bounding the occupied floor area:

$$\sum_{h,\,r\in R(h)}\gamma_{gr}\,n_{hr}\,\overline{L}_{hr}\ \ge\ \alpha\,\Gamma_K\,(Z-1+\theta),\qquad \sum_{h,\,r\in R(h)}\hat{w}_{gr}\,n_{hr}\,\overline{L}_{hr}\ \ge\ \alpha\,pK^w\,(Z-\theta), \tag{71}$$

$$\sum_i\sum_h p_hq_hX_{ih}\ \le\ pK\cdot qK,\qquad \sum_i\sum_h p_hq_hX_{ih}\ \ge\ \alpha\,pK\cdot qK\,(Z-1+\theta). \tag{72}$$

(71) is (35) with the block index dropped. (72) is a valid inequality rather than a transcription, and is where the footprint consistency of Section 4.7 earns its place: its first row says the footprints lie on the floor, and for the second, multiplying (63) by p_hq_h and using $n_{hr}\gamma_{gr}=p_hq_hr_{gr}$ gives

$$\sum_{r\in R(h)}\gamma_{gr}\,n_{hr}\,L_{ihr}\ =\ p_hq_h\sum_{r\in R(h)}r_{gr}\,L_{ihr}\ \le\ p_hq_h\,rK\,X_{ih},$$

so the volume the stacks hold is at most rK times the occupied area; summing over i and h and chaining through (71) gives the row, and a truck full by volume therefore has at least a fraction α of its floor occupied. Without the tiling assumption the step fails and the row can cut off feasible layouts.

7.1 The direction-specific rows

Plant-bound, $g=\mathrm{p}$, with $\mathrm{image}_n=\overline{V}_n$:

$$\sum_{n\in M(r)}V_{in}\ \le\ \sum_{h\,:\,r\in R(h)}n_{hr}\,L_{ihr} \qquad \forall i\in\mathcal{I},\ r\in R \tag{73}$$

$$V_{in}\ \le\ \bar{u}^V_n\,Z \qquad \forall i\in\mathcal{I},\ n\in N \tag{74}$$

$$\sum_{n\in N}w_n\,\overline{V}_n\ \le\ pK^w\,Z \tag{75}$$

$$\sum_{n\in N}w_n\,V_{in}\ \le\ W^{\max} \qquad \forall i\in\mathcal{I} \tag{76}$$

and supplier-bound, $g = \mathrm{s}$, with $\text{image}_r = V^{\mathrm{e}}_r$:

$$V^{\mathrm{e}}_r \;\le\; \sum_{h\,:\,r\in R(h)} n_{hr}\,\overline{L}_{hr} \qquad \forall r \in R \tag{77}$$

$$V^{\mathrm{e}}_r \;\le\; \bar{u}^{\mathrm{e}}_r\,Z \qquad \forall r \in R \tag{78}$$

$$\sum_{r\in R} w^{\mathrm{f}}_r\,V^{\mathrm{e}}_r \;\le\; pK^{w}\,Z \tag{79}$$

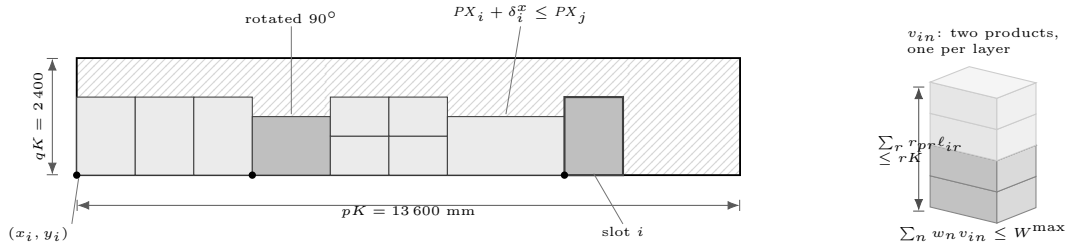
$$\sum_{h}\sum_{r\in R(h)} w^{\mathrm{f}}_r\,n_{hr}\,L_{ihr} \;\le\; W^{\max} \qquad \forall i \in \mathcal{I} \tag{80}$$

The asymmetry is the compact model's own: the crush limit on a plant-bound truck depends on which products ride in the slot, so it needs the per-slot assignment V_{in}, and a supplier-bound truck does not.

What the method requires of the oracle. The oracle's only obligation is to return an optimal point of *this* feasible set for the duals it is handed, and three conditions make its answer usable. The slot set must be the true bound (46), an optimum proved over fewer slots bounding a different problem. The floor must be imposed on the load carried, with each commodity's own weight. And the block must accept a per-commodity cap on the image, which is what Section 8.1 asks of it: an oracle that silently ignores such a request answers a different question. The hard core is the disjunctive relation (64)–(66), whose linear relaxation is weak, and which may equivalently be imposed as one two-dimensional disjointness relation over the slot rectangles with no separating binaries at all.

The volumetric oracle. The same master is also solved over the loads of the volumetric benchmark of Appendix A, where a load is a column iff its volume fits the trailer's, its weight the payload, and it clears the floor on the binding resource. Pricing is then a two-constraint integer knapsack, which proves optimality routinely and makes a certificate affordable at every node. Running both oracles through one master makes Section 9.5 an experiment about the loading representation rather than about two solution technologies.

Improving, or optimal. Any column of negative reduced cost improves the restricted master; the *most* negative one is needed only for the certificate, so the two uses are priced differently. A sweep that may add columns passes the pair's threshold to the oracle as an objective limit, and a call that returns no point is then a proof that the pair has no improving column rather than a failure to find one. A sweep that certifies prices every pair to proven optimality, under no objective limit and no early stop, which is what Algorithm 1 requires. What separates them is cost, not validity: one sweep is $2|T|$ calls, one per count row, and a pair that has an improving column can spend its whole per-call budget proving that column optimal.



(a) the floor, (64)–(67) (b) one slot, (49) and (51)

Figure 4: What the pricing problem designs. A column is a drawing a loading crew could execute: footprints placed on the floor without overlap and inside $pK \times qK$, each slot a stack of homogeneous layers under the height budget (63) and the per-slot crush cap, the load assigned to the slots in the same period so that $b_{\omega n} = \sum_{i\le m} v_{in}$, and the whole truck full on the binding resource (69)–(70): volume *or* payload, $\max\{\gamma/\Gamma_K,\, w/pK^{w}\} \ge \alpha$, and never the sum of the two ratios.

What pricing cannot return. For any dual vector (60) is linear in the image, so it attains its optimum at an extreme point of $\mathrm{conv}(\Omega^g)$: pricing returns extreme loads only. A balanced load inside the hull is dominated componentwise by a fractional blend of single-commodity trucks — the relaxation buys that

blend for one truck where an integer plan pays for each truck it mixes — so no choice of duals returns it, however the search moves them (Figure 5). Interior loads are precisely what an integer plan needs when a period's demand does not decompose into full trucks, which is why the method does not ask pricing for them. The flow-first construction of Section 8.2 reaches the interior directly, and branching on the fleet aggregates withdraws the dominating column, so the duals move and the oracle is asked for a load the pool does not yet hold.

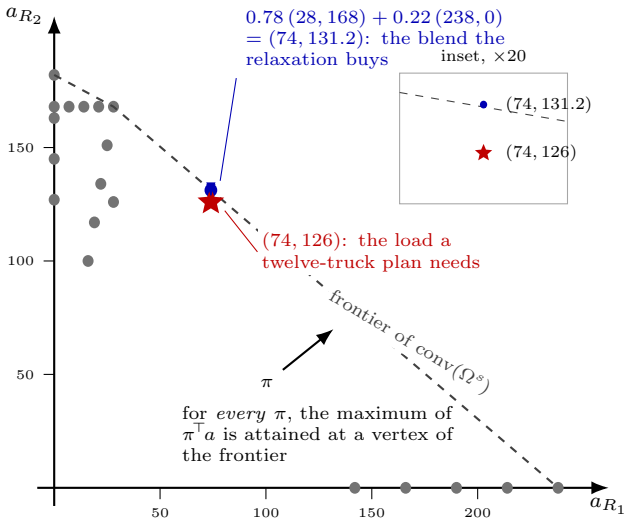


Figure 5: Why the load a cheaper plan needs is never priced. Dots are the supplier-bound loads obtained from a sweep of demand-derived price vectors; the dashed line is the upper-right frontier of their convex hull. The pricing objective is linear, so for *any* dual vector π its optimum is attained at a vertex of that frontier. The load $(74, 126)$ lies strictly below it — it is dominated componentwise by the blend $0.78\,(28, 168) + 0.22\,(238, 0) = (74, 131.2)$ — so no choice of duals returns it. The relaxation buys that blend for one truck, while an integer plan must pay for both of the trucks it mixes. The construction of Section 8.2 reaches the interior directly: it partitions a delivery vector into legal loads instead of asking a linear objective for them.

8 Branching, with the flow-first construction inside it

The branching hierarchy is read off the robustness criterion of Section 6.3. A row with $\phi \equiv 1$ — any bound on a sum of dispatch counts, including the per-class total n_{gt} and the fleet $X = \sum_{g,t} n_{gt}$ — has its dual folded into μ_{gt} exactly as (57) does, and all such aggregates are integer in every plan. A row with ϕ affine in the image, a bound on $\overline{V}_{nt}$ or $\overline{V^{\mathrm{e}}}_{rt}$, is absorbed into π_{ct}, already an input of the pricing problem; these count physical RTIs, so they too may carry a dichotomy. Bounding a single $\lambda^g_{\omega t}$ is neither: honouring it means excluding that column from the subproblem (Vanderbeck 2000), destroying the property that one column set serves every period. It is also useless here, the master holding many columns that differ in a single RTI, so the relaxation answers by shifting the same fractional weight one column over.

Between the two robust rungs the first binds, because transport cost is $cK \cdot X$ and nothing else in the objective counts trucks. A bound on a delivery is answered by moving freight between periods at the price of one period's holding cost, and so is a bound on a *single* period's dispatch total; the aggregate with no such escape is the horizon total. We therefore branch on the fleet, in two levels (Figure 6), keeping the delivery rung as a third entered only where the first two have nothing left to say — which, in the runs reported here, is nowhere: it is disabled throughout, as Section 9.1 records. Master, pricing problem, construction and tree are together the procedure we call BP-3D-SCPTOP (Algorithm 2).

Level 1: the total fleet. Given an incumbent with fleet vector $\mathbf{n}^\star$ and total $X^\star$, three regions partition the space. $X \geq X^\star + 1$ costs a whole truck with no offset and is usually pruned by bound; $X = X^\star$ fixes transport, so everything inside it is inventory allocation and this is where level 2 goes; and $X \leq X^\star - 1$ is

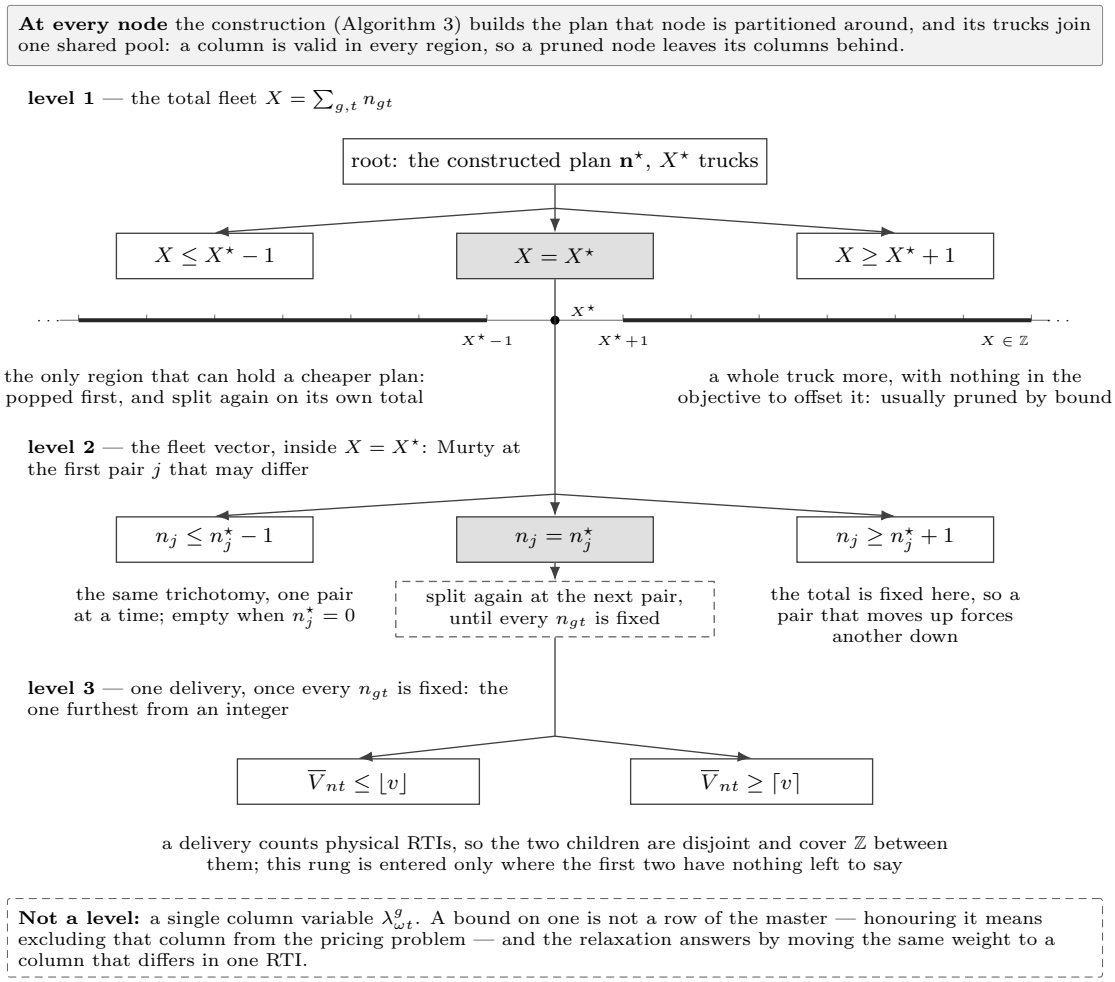


Figure 6: Every rung bounds an integer quantity the master already prices, so the children of a split are disjoint and together exhaustive and the search gives up nothing by taking them. The order is forced by the objective: transport costs $cK \cdot X$ and nothing else in the model counts trucks, so the horizon total is the one aggregate that cannot be answered by moving freight between periods for one period's holding cost, as a single dispatch count or a single delivery can. And because these are rows of the master and not restrictions on the subproblem, branching moves the duals and leaves the pricing problem alone — which is also how a load no dual vector would have asked for (Figure 5) comes to be priced at all.

the only region that can beat the incumbent outright, so it is worth a certificate of its own and the stack is ordered to expand it first. A plan found *inside* an inequality region splits it on its own total the same way, so level 1 recurses.

Level 2: the fleet vector, inside $X = X^\star$. Fix an order on the pairs (g, t). The fleet vector of any plan in the region differs from $\mathbf{n}^\star$ at a first index j, or nowhere, so branching there gives three children,

$$n_j \le n_j^\star - 1, \qquad n_j = n_j^\star, \qquad n_j \ge n_j^\star + 1, \tag{81}$$

disjoint and exhaustive, with the equality child partitioned again at $j + 1$ (Murty's scheme (Murty 1968)) and expanded first. The partition is taken around the plan *constructed at that node*, so a node that improves the incumbent is re-partitioned around the better plan at once and nothing is restarted.

Pruning, and what counts as a proof. A node is a region imposed on the RMP as fleet rows. Its LP over the current pool is solved first as a filter (Section 6.4); where the filter allows pruning and the

certification policy admits the node, column generation is run to a certificate, and a certified value at or above the incumbent prunes it while a value below becomes that region's bound. Every other node carries its parent's, valid because the child's region is a subset. An infeasible node relaxation is *not* a proof that a region is empty: stock non-negativity is not elastic, so a fleet lower bound can force deliveries up through whatever columns the pool holds and kill the relaxation for a reason internal to the pool. Only arithmetic licenses that sentence, and three such arguments are available: the fleet box intersected with the interval on X, an availability cap telescoped from the sending rows (11) and (13) under non-negative stock, and the constants of (59) read as a requirement floor.

What the tree reports. The incumbent is the cheapest verified plan any construction found. The global bound is the minimum over the *frontier*, not the root, a region nobody expanded contributing nothing; at the deadline the open regions are swept and certified where the filter allows. Under 3D loading a region's certificate comes, like the root's, from the volumetric relaxation with the region's fleet rows applied, which Proposition 2 covers because those rows bound the fleet vector and not the load model. A column, by contrast, is a feasible load, so one found in any region is valid in every other and a pruned node leaves its columns behind. Optimality is claimed *only* when the root is certified, no region remains open, no restart has broken the partition and the incumbent meets the global bound. The hierarchy is finite and its levels exhaustive, so those conditions are reachable in principle; no run here meets them.

Two levels of the three ran. The delivery level is implemented and was left disabled in every run reported here, so what covers a full-depth Murty leaf is a certificate rather than a further split — and a leaf whose certificate sits below the incumbent is then bounded but not refuted, however much budget remains. That is a property of the configuration and not of the procedure: with the delivery level on, such a leaf is split again. It is also not what stopped the runs of Tables 7 and 8, which ended with between two and fifteen regions still on the stack, i.e. on their budget and not on the hierarchy.

8.1 The initial pool

The search starts from the plainest possible pool: one maximally full truck per commodity and direction, obtained geometrically by pricing that commodity alone with every other capped at zero — so that where the probe closes, the pool truck is that commodity's placeable maximum and the same number is its placeability cap — and volumetrically by filling to capacity. No demand mixes and no reduced variants are seeded, so whatever the search reaches it reached by pricing, branching and construction.

8.2 The flow-first construction

The primal heuristic decides first how much moves and how many trucks carry it, and builds the trucks afterwards. It runs inside the region $\mathcal{R}$ of the node that calls it and must stay there, or the children that node induces would partition around a point it excludes, and it runs on a *fresh copy* of the restricted master, step 2 changing variable types the node's own LP needs intact. Figure 7 follows one period through the four steps of Algorithm 3.

1. *Restricted master.* The LP over the pool, inside $\mathcal{R}$, gives the duals and the fractional point.
2. *Aggregate integer master.* Introduce the dispatch totals n_{gt} as explicit integer variables, make $\overline{V}_{nt}, \overline{V^{\mathrm{e}}}_{rt}$ integer and leave the column variables *continuous*, with $\mathcal{R}$ imposed on the n_{gt} as well as on the columns and, under 3D loading, the aggregate forms of the rows (87)–(89) added (Section 8.3). The objective is decided here, and *without a single call to a pricing problem*: those rows aside, the row set is the master's own. Artificial slack above a tolerance means the region is infeasible over this pool, and the node is abandoned.
3. *Partition, as a cover.* For each direction and period, realise the integer deliveries $Q^{\star}_{gt}$ as $n^{\star}_{gt}$ legal

Algorithm 2 BP-3D-SCPTOP: branch and bound over the fleet vector, with CONSTRUCT as primal heuristic and CERTIFY as bound.

$\bar{\Omega} \leftarrow$ one maximally full single-commodity truck per commodity and direction, each verified ▷ no demand mixes, no reduced variants
$\mathrm{LB}_0 \leftarrow \textsc{Certify}(\bar{\Omega}, \mathrm{root})$; $(\bar{z}, \mathbf{n}^\star, \bar{\Omega}) \leftarrow \textsc{Construct}(\bar{\Omega}, \mathrm{root})$; $X^\star \leftarrow \sum_{g,t} n^\star_{gt}$
$\mathcal{S} \leftarrow [\, X \ge X^\star + 1,\ X = X^\star,\ X \le X^\star - 1\,]$ ▷ a stack: the region that can beat the incumbent is popped first
while $\mathcal{S} \neq \emptyset$ and time remains **do**
  $\mathcal{R} \leftarrow$ pop $\mathcal{S}$; impose it on the RMP as fleet rows
  if $\mathcal{R}$ is refuted arithmetically **then**
    continue ▷ fleet box, availability, requirement
  end if
  $\tilde{z} \leftarrow$ LP over $\bar{\Omega}$ inside $\mathcal{R}$ ▷ over-estimates the region's LP: a filter, never a bound
  if $\tilde{z} \ge \bar{z}$ and the policy certifies $\mathcal{R}$ **then**
    $(\mathrm{LB}_{\mathcal{R}}, \mathrm{proved}) \leftarrow \textsc{Certify}(\bar{\Omega}, \mathcal{R})$
    if $\mathrm{LB}_{\mathcal{R}} \ge \bar{z}$ **then**
      continue ▷ pruned
    end if
    record $\mathrm{LB}_{\mathcal{R}}$ as an open bound
  else
    $\mathcal{R}$ carries its parent's bound
  end if
  $(z, \hat{\mathbf{n}}, \bar{\Omega}) \leftarrow \textsc{Construct}(\bar{\Omega}, \mathcal{R})$ ▷ its columns join the shared pool whatever it returns
  if no plan **then**
    price under $\mathcal{R}$'s duals and construct once more; **continue** if still none
  end if
  if $z < \bar{z}$ **then**
    $(\bar{z}, \mathbf{n}^\star) \leftarrow (z, \hat{\mathbf{n}})$ ▷ new incumbent
  end if
  if $\mathcal{R}$ does not fix X **then**
    push the three regions of level 1 around $\sum_{g,t} \hat{n}_{gt}$ ▷ level 1 recurses
  else if the Murty depth j is below the cap **then**
    push the three children (81) at the j-th pair, around $\hat{\mathbf{n}}$, equality last
  end if
end while
for all $\mathcal{R}$ still on $\mathcal{S}$, while a reserve of time remains **do**
  filter by the pool LP; certify the survivors; record their bounds ▷ the frontier
end for
$\mathrm{LB} \leftarrow \max\{\mathrm{LB}_0, \min$ over the open and frontier bounds$\}$, capped at $\bar{z}$
verify every dispatched truck of the incumbent against (48)–(53)
return $(\bar{z}, \mathrm{LB})$; optimality is claimed only if LB_0 was proved, $\mathcal{S} = \emptyset$, no restart occurred and $\bar{z} = \mathrm{LB}$

trucks by an integer programme over per-truck commodity counts a_{kc}, $k \le n^\star_{gt}$:

$$\min \ \sum_c \gamma_{gc}\Big(\sum_k a_{kc} - Q^\star_{gtc}\Big) \tag{82}$$

$$\text{s.t.} \ \sum_k a_{kc} \ \ge \ Q^\star_{gtc} \qquad \forall c \tag{83}$$

$$\textstyle\sum_c \gamma_{gc} a_{kc} \le \Gamma_K, \qquad \sum_c w_c a_{kc} \le pK^w \qquad \forall k \tag{84}$$

$$\textstyle\sum_c \gamma_{gc} a_{kc} \ge \alpha\Gamma_K \theta_k, \qquad \sum_c w_c a_{kc} \ge \alpha p K^w (1-\theta_k) \qquad \forall k \tag{85}$$

$$\textstyle\sum_c w_c a_{kc} \ \ge \ \sum_c w_c a_{k+1,c} \qquad \forall k < n^\star_{gt} \tag{86}$$

$$a_{kc} \in \{0, \dots, \bar{u}_{gc}\}, \qquad \theta_k \in \{0,1\}.$$

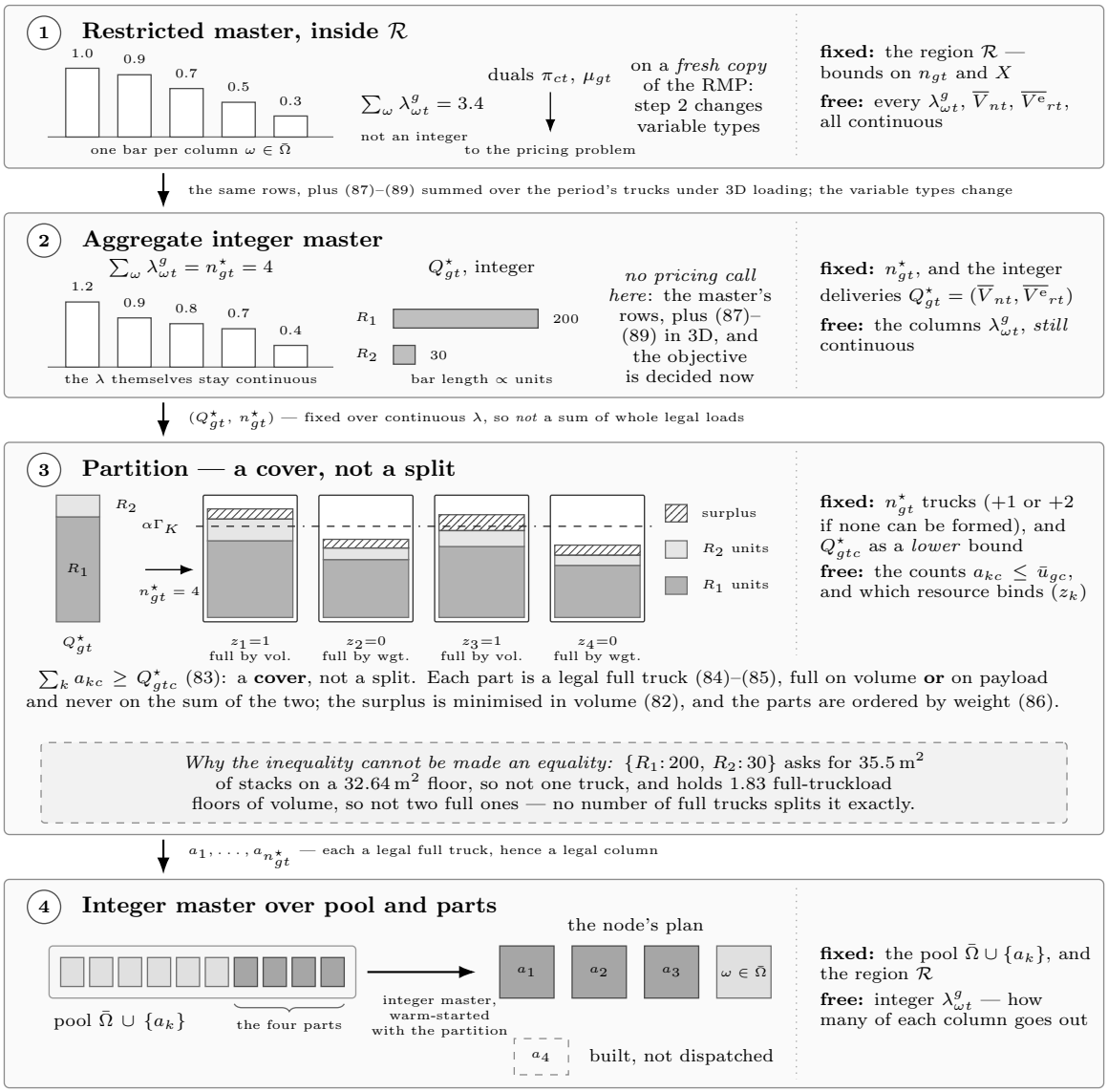


Figure 7: The flow-first construction over one period, and why its third step covers rather than splits. Steps 1 and 2 settle how much moves and how many trucks carry it while the column variables stay *continuous*, so the delivery vector they fix need not be a sum of whole legal loads — for the vector shown, no number of full trucks realises it exactly. Step 3 therefore asks only that the parts *cover* it (83), paying for the hatched surplus (82), and makes each part full on volume *or* on payload (85) rather than on the sum of the two. Nothing built is binding: the parts enter step 4 as columns with a warm start, and the integer master may dispatch fewer of them than the cover produced.

(84)–(85) make each part a legal truck, θ_k selecting the resource on which part k meets the floor; (86) orders the parts by weight; $\bar{u}_{gc}$ is the placeability cap below. If $n^\star_{gt}$ parts cannot be formed, $n^\star_{gt} + 1$ and $+2$ are tried where $\mathcal{R}$ allows — and a timeout is not a proof that none can be.

4. *Integer master over pool plus parts.* The parts join the pool as columns and the integer master is solved over it, warm-started with the partition; its solution is the node's plan.

Why the partition must be a cover. This is the step at which the procedure could quietly become wrong. With integer λ the aggregate $\sum_\omega b_{\omega n} \lambda^p_{\omega t}$ is a sum of placeable loads, so an unrealisable delivery vector cannot be requested; step 2 breaks that link by fixing the aggregates over *continuous* λ. A fractional

blend of legal loads need not be realisable as legal loads for *any* number of trucks: floor area is consumed in whole footprints while (85) is measured in volume, so splitting a vector into more trucks consumes more floor without adding volume and splitting it into fewer violates the trailer. On one benchmark instance $\{R_1 : 200, R_2 : 30\}$ holds 1.83 full-truckload floors of volume but needs 35.5 m^2 of stacks on a 32.64 m^2 floor. A cover is sound because every part is a legal column and the parts enter step 4 *as columns with a warm start*: the integer master may dispatch fewer than were built, and over-delivering is feasible, stock having no upper bound.

Algorithm 3 CONSTRUCT($\bar{\Omega}, \mathcal{R}$) — the flow-first relax and fix. Line 4, the rows of line 12 and lines 15–27 apply under 3D loading only. ε_2 is step 2's relative MIP gap and *tol* the tolerance above which artificial slack means the region admits no plan over this pool.

1: $\bar{\Omega}' \leftarrow$ a fresh copy of the RMP over $\bar{\Omega}$ ▷ not the node's own LP: step 2 changes variable types
2: solve LP($\bar{\Omega}', \mathcal{R}$) ▷ duals π, μ and the fractional point the node stands on
3: make $n_{gt}, \overline{V}_{nt}, \overline{V^{\mathrm{e}}}_{rt}$ integer; keep λ continuous; impose $\mathcal{R}$ on n_{gt} and on λ
4: add the aggregate forms of (87)–(89) ▷ 3D only; layer coefficient = the height count, the crush limit being its own row
5: solve to relative gap ε_2
6: **if** no solution, or artificials > tol **then**
7: **return** $\perp$ ▷ node abandoned; the region is infeasible over this pool
8: **end if**
9: $(Q^\star, n^\star) \leftarrow$ the integer deliveries and fleet
10: **for all** $(g, t) \in G \times T$ **do**
11: **for** $m = n^\star_{gt}$, $n^\star_{gt} + 1$, $n^\star_{gt} + 2$ while $\mathcal{R}$ allows **do**
12: solve PARTITION($Q^\star_{gt}, m$), (82)–(86), with $\bar{u}_{gc}$ as variable bounds and, in 3D, (87)–(89) per part
13: **break** if feasible ▷ a timeout is not a proof of infeasibility
14: **end for**
15: **for all** parts a_k of the cover **do**
16: (verdict, layout) $\leftarrow$ PACK(a_k)
17: **if** placeable **then**
18: verify the layout; $\bar{\Omega} \leftarrow \bar{\Omega} \cup \{(a_k, \text{layout})\}$
19: **end if**
20: **if** *proved* unplaceable **then**
21: bisect $\sigma \in (0, 1]$ and cut every load dominating $\lfloor \sigma a_k \rfloor$ ▷ validity
22: **end if**
23: **if** undecided **then**
24: exclude a_k as a single point ▷ heuristic restriction, never validity
25: **end if**
26: **end for**
27: re-solve the partition if a cut was written and rounds remain
28: **end for**
29: $\bar{\Omega} \leftarrow \bar{\Omega} \cup \{\text{the parts}\}$; solve the integer master over $\bar{\Omega}$, warm-started with the partition
30: **return** $(z, \hat{\mathbf{n}}, \bar{\Omega})$ ▷ the parts are kept even when no plan is returned

8.3 The construction under 3D loading

Rows (84)–(85) are exactly the volumetric column conditions, so over that oracle every part is a column by construction. Under 3D loading a part must also satisfy (48)–(53); since the volumetric conditions are *necessary* for geometric feasibility (Proposition 2) the partition programme is kept as a *relaxation* of the geometric one, and each part is then tested for placeability. Step 2 makes no oracle call, so the objective is still decided before any part is put to the placeability oracle; what step 2 sees of the geometry is the necessary rows at the end of this section.

Everything here rests on one property. Placeability is **downward closed**: given a layout of $a \geq b$, removing the surplus units from their slots leaves positions unchanged, so nothing overlaps, every stack is

no taller and no heavier, and an emptied slot is dropped. The full-truckload floor is no part of placeability, so it does not disturb the closure.

Deciding placeability. A part is submitted to an oracle that returns one of three answers. A constructive *shelf design* — a small integer programme laying the part's stacks in shelves along the trailer, one integer count per shelf, footprint and orientation — supplies most positive answers (93% on the ladder of Section 9.3), and its layout passes the independent verifier of Section 9.1 before it counts. Infeasibility proofs come from decoupling: a stack carries one footprint class only, so a load within the payload is placeable if and only if the multiset of $\underline{s}_f$ rectangles of each class f can be placed on the floor, $\underline{s}_f$ being the least number of class-f stacks its units need — any layout has at least that many, and discarding rectangles from a placement leaves one. The least stack count is a small cutting-stock problem per class, and whether the rectangles fit is decided exactly by a position-indexed placement model over the normal patterns of the floor (Christofides and Whitlock 1977; Beasley 1985), after the arithmetic conditions below — payload, area and (88)–(89), at the least stack counts they allow — have had the chance to refute the load at no cost. Since unplaceability is monotone in the stack counts, an infeasible placement at a *lower* bound on $\underline{s}_f$ already refutes; a positive answer counts only where the stacking attains $\underline{s}_f$ and the layout it lifts to passes the verifier.

Two kinds of cut, which must never merge. A part *proved* unplaceable yields a *dominance cut* forbidding every load that dominates it componentwise, valid by the closure above, so one cut removes the whole up-set of the refuted load — including every point of the one-unit-at-a-time staircase the partition would otherwise walk down. Before cutting, the load is tightened along its own ray: $\lfloor \sigma a \rfloor$ is probed for σ bisected in $(0, 1]$ and the cut written at the smallest scaled point still *proved* unplaceable. A part left *undecided* is instead excluded as a single point so that the partition proposes something else: a restriction of a primal heuristic, never a validity claim. Reading an undecided verdict as a dominance cut is how a method comes to forbid its own optimum.

Placeability caps. The largest number of one commodity a truck can hold is a scalar and, by the same closure, monotone, so it is computed rather than walked to: bisected once per direction and commodity at the start of a run, a proved optimum tightening it and an undecided probe leaving the analytic cap, then handed to the partition as $\bar{u}_{gc}$.

A stack relaxation inside the partition. The partition additionally carries, per truck k and footprint class f — a stack type h of Section 4, with $p_f = p_h$ and $q_f = q_h$ — an integer number of stacks $s_{kf} \le \lfloor pK \cdot qK / p_f q_f \rfloor$, the crush limit and the height budget imposed per stack and the stacks' areas summed against the floor:

$$\sum_{c \in f} w_c a_{kc} \le W^{\max} s_{kf}, \qquad \sum_{c \in f} r_{gc}\, a_{kc} \le rK\, s_{kf}, \qquad \sum_{f} p_f q_f\, s_{kf} \le pK \cdot qK, \tag{87}$$

and step 2 carries the same rows per (direction, period) with the area right-hand side multiplied by n_{gt}. Both are necessary, every truck of the period having integer stack counts that satisfy the per-truck rows, and they are what the volumetric relaxation omits. Where a class carries one RTI type the height row sharpens to $\sum_{c \in f} a_{kc} \le \lfloor rK / r_{gr} \rfloor s_{kf}$: the crush limit is already carried by the first row with each commodity's own weight, so the height row takes the height-limited count, which is what the runs reported here used; the smaller of the two counts is also valid, and strictly tighter. These aggregates remain a relaxation of the per-truck split, which is why step 3 keeps its $n + 1$ fallback.

Necessary conditions of placeability as rows. The rows (87) are the coarsest of the conditions by which the oracle refutes a load, and the partition carries the finer ones too, so that it stops proposing parts the oracle would refuse. With C_f the commodities of footprint class f in direction g and $L_f^{\max}$ the most units of them one stack can hold — a two-resource knapsack under rK and $W^{\max}$ over all of C_f — each truck k and class f satisfy

$$\sum_{\substack{c \in C_f \\ w_c \ge \xi}} a_{kc} \le \left\lfloor \frac{W^{\max}}{\xi} \right\rfloor s_{kf} \;\; \forall \xi \in \{w_c\}_{c \in C_f}, \qquad \sum_{\substack{c \in C_f \\ r_{gc} \ge \xi}} a_{kc} \le \left\lfloor \frac{rK}{\xi} \right\rfloor s_{kf} \;\; \forall \xi \in \{r_{gc}\}_{c \in C_f},$$
$$\sum_{c \in C_f} a_{kc} \le L_f^{\max} s_{kf}, \tag{88}$$

the height thresholds being written only where a class carries more than one RTI type. On the floor, split $s_{kf} = s^0_{kf} + s^1_{kf}$ into integer counts of stacks laid with p_f along the trailer's length pK and along its width qK ($s^1_{kf} = 0$ for a square footprint); then

$$\sum_f \big(u_1(p_f)\, q_f\, s^0_{kf} + u_1(q_f)\, p_f\, s^1_{kf}\big) \le U_1\, qK, \qquad \sum_f \big(p_f\, u_2(q_f)\, s^0_{kf} + q_f\, u_2(p_f)\, s^1_{kf}\big) \le pK\, U_2, \tag{89}$$

for every pair (u_1, U_1) on pK and (u_2, U_2) on qK of a finite family of *dual-feasible functions*: (u, U) is one on a length C when $\sum_i x_i \le C$ implies $\sum_i u(x_i) \le U$ (Fekete and Schepers 2001; Clautiaux, Alves, and Valério de Carvalho 2010). The family is the identity with $U = C$, which returns the area row of (87); the thresholds $\lceil x \ge \ell \rceil$ at the footprint lengths ℓ that occur, with $U = \lfloor C/\ell \rfloor$; and the scalings $\lfloor jx/C \rfloor$ with $U = j$, for $j \le 12$, valid because the floor function is superadditive.

Every row holds for every placeable truck once s_{kf} and its split are read off the truck's own layout. A stack bears at most $W^{\max}$, so it holds at most $\lfloor W^{\max}/\xi \rfloor$ units weighing ξ or more; the height threshold is the same argument on rK, and $L^{\max}_f$ bounds what any stack of the class holds. For (89), a line across the floor parallel to its length meets slots whose extents along it are disjoint and sum to at most pK, so their u_1-values sum to at most U_1; integrating along the width gives the first row, and exchanging the axes the second — the conservative scales of Fekete and Schepers (2004) with one dimension left unscaled. Step 2 carries their sum over the period's trucks: the deliveries $\overline{V}_{nt}$ or $\overline{V^{\mathrm{e}}}_{rt}$ in place of a_{kc}, integer stack and orientation counts summed over the trucks, and the right-hand sides of (89) times n_{gt}. They are implied wherever every point is already a placeable truck: in the pricing problem, whose integer points are layouts — whether they would tighten its weak relaxation, as (72) does, is untested — and in the master over placeable columns, each of which satisfies them, so that they hold for every non-negative combination of columns. They are *not* implied in the volumetric relaxation that supplies every bound reported here, whose columns are not layouts; Section 10 returns to that. In the method as run they cut only where the load is a free integer vector: a part of step 3, or a delivery vector of step 2 over continuous column weights.

9 Computational results

9.1 Protocol, instances and what each quantity means

The method is implemented in Julia and solves every model with Gurobi 13; the compact models are solved with Gurobi 12. The placeability checks and re-packs of Table 10 were computed with an earlier, Python implementation of the method's partition and placeability oracle, which that table and Section 9.5 call the first implementation. Table 5 declares, per stage, the five parameters that can change a solver's answer: time limit, MIP gap, threads, per-model memory cap and seed. Memory is part of the budget — Gurobi holds its tree in memory — so the time at which a fixed cap is reached licenses no inference about difficulty. Every row below is read from a recorded run, each one declaring its own budget, thread count, memory cap and commit. A *gap* is always $(z - \mathrm{LB})/z$ against the method's own certified bound; Table 6 defines the rest.

Every row is one run, and one run is reproducible: the procedure is deterministic given a build, a machine and the way its code is executed. It is not deterministic across those, and that is a property of the method rather than of any implementation of it. Every phase draws on a wall-clock budget, so which phases finish, how many construction attempts fit and which regions the tree opens all depend on execution speed; a plan is the method's answer under one schedule and not a function of its configuration alone. Every run reported here was executed the same way, and every row records the source tree it ran from and how that code was loaded, so the tables are internally comparable. Two consequences are taken seriously throughout: a difference of one or two trucks between rows is not read as an effect, and no claim rests on one.

Three settings that are not budgets also decide what the tables can say. Certificates are computed at the root, at the one region that can refute the incumbent outright — "is there a plan with one truck fewer" — at a full-depth leaf, which cannot be split (below), and in the frontier sweep at the deadline; under 3D loading every certificate is the volumetric one (Section 8). Certifying every node closes far more of the space, so the *open* counts below are a property of that choice and not of the method; the two modes are reported under one policy so that their columns mean the same thing. The second is that the branching hierarchy's third level — the deliveries — is disabled in every run reported here, so a full-depth leaf is covered by a certificate rather than split again; Section 8 says what that costs and what it does not. The third is that no seed is set by the implementation of Table 7 and the size ladder; the solvers' defaults apply.

Instances. Instances are named $I_|N|_|T|_|R|_v$, with v the number of erected RTIs that fill one truck by volume. Footprints come from the industrial catalogue, from 800×600 to 2400×1200 mm, with heights

Table 5: Declared budgets, one line per stage that produces a number reported below. *gap* is the relative MIP gap at which an integer solve may stop and *mem* the per-model soft cap. Every line is read from the run files, except the method's solver version, read from its package manifest. The volumetric parity runs declare their own configuration in each row, which is what lets Table 7 compare like with like in budget, threads and cap, under Gurobi 13 against 12. No seed is set by the method's implementation, so the solver's default applies. On the size ladder the construction carries the rows (88)–(89) in steps 2 and 3, and step 4's integer master is limited, per call, to one sixth of the rung's budget. The ladder's other per-call limits are the code's defaults: step 2 300 s at the root and 120 s at a node, at relative gap $\varepsilon_2 = 10^{-4}$; partition 60 s; pricing and placeability 20 s each; root column generation 15% of the budget and the frontier sweep another 15%. The method's volumetric pricing models carry a 1 GB soft cap, checked against the memory of the Gurobi environment they share, not the 20 GB declared; every bound it reports came back proved.

Stage	solver	time (s)	gap	threads	mem (GB)	seed
BP-3D-SCPTOP, size ladder	Gurobi 13	900–3 600	—	10	20	not set
step 4, per call	Gurobi 13	budget/6: 150–600	—	10	20	not set
Compact SCPTOP (volumetric)	Gurobi 12	600–3 600	0	10	20	0
Compact 3D-SCPTOP, $n_{\mathrm{LC}} = 3$	Gurobi 12	1 800	0	10	20	0
BP-3D-SCPTOP, volumetric parity	Gurobi 13	3 600	—	10	20	not set
Placeability check, Table 10	shelf + CP-SAT	60 per load	—	—	—	—

Table 6: What each reported quantity means. A sentinel recorded by a run that obtained no bound is never printed, never averaged and never rounded to a side.

z	The cost (54) of the plan returned, net of any artificial penalty; a plan is reported only if it carries no artificial slack. An upper bound on the optimum in every case.
trucks	$\sum_{g,t} n_{gt}$ for that plan.
LB	A lower bound on the optimum of the model the method solves. For the method it is the *certified* bound of Section 6.4, printed only where the sweep behind it came back with every pair proved. For a compact model it is the solver's dual bound at its stop.
gap	$(z - \mathrm{LB})/z$, against the method's own bound. An optimality gap only to the extent that bound is tight.
stop	How a solve ended: OPTIMAL, TIME_LIMIT or MEM_LIMIT. A time or memory stop makes the row's time the budget rather than a solve time.
nodes / open	Tree nodes opened; regions left unexpanded at the deadline. A run with open regions has not searched the space, whatever its gap — and a run with none has not refuted it either, a region closed with a bound below the incumbent being bounded and not excluded.
dropped	Regions popped and abandoned without a certificate of their own, for instance because the construction returned no plan in them; only the root's bound covers them.
verified	Dispatched trucks re-derived from the column's own fill and found legal by the independent routine of Section 9.1, out of the number dispatched.
Δz	The cost of one method relative to another on the same instance, with the denominator named where it appears.

quantised to 50 mm and bounded so that at least two layers fit under the roof; demand deviates ±10% from its mean, and content weight by ±25% between products sharing a type, which is what makes the crush limit depend on the period's mix. Every instance here has $|R| = 2$ and $v = 90$. The smallest, $I_8_5_2_90$, consumes roughly 2.5 truckloads per period; the others cross $(|N|, |T|) \in \{(8, 10), (12, 12), (16, 15), (24, 20)\}$ with 2.5, 5 or 8 truckloads per period, the name's suffix being that figure times ten, not K. The truck is a semi-trailer, 13600 × 2400 × 3000 mm, payload 24 t, $W^{\max} = 1200$ kg, $\alpha = 0.55$, $cK = 1000$ per dispatch, $lt = 1$, one period of coverage stock.

Verification. Every plan below is re-derived from its dispatch list alone and checked truck by truck against (48)–(53) by a routine independent of the search and of the oracle that built the column, re-deriving each slot's weight from the fill the slot carries and rejecting a fill that does not describe the stack it sits in. It shares the full-truckload predicate with the oracle and the partition floor, so it catches a defect in a solver's *satisfaction* of that definition and not one in the definition.

9.2 What the volumetric side gives

Table 7: BP-3D-SCPTOP under its *volumetric* oracle against the compact SCPTOP formulation, both solving the *volumetric* problem of Appendix A, so that the two bracket one optimum. **This table is a check on the procedure, not a contest**: the volumetric problem is the one case here where an independent optimum exists, and the question each row answers is whether the procedure reaches it. z is the incumbent at the stop; OPT marks a proved optimum, ^{m} a stop on the memory cap, ^{t} a stop on time. "bound" is Gurobi's dual bound for the compact model and the certified master-LP bound of Section 6.4 for the method. Δz is $(z_{\mathrm{DW}} - z_{\mathrm{compact}})/z_{\mathrm{compact}}$; negative means the method's plan is cheaper. Both sides had 3 600 s, 20 GB and ten threads — except that the compact model was given 1 800 s on $I_8_5_2_90$, where it proved the optimum in 0.2 s and the budget never bound. The method's wall clock stops short of 3 600 s by construction: the search is given $(1 - 0.15) \times 3\,600 = 3\,060$ s and the rest is reserved for the frontier sweep that bounds the regions left open, which returns here in 2 to 73 s rather than spending its reserve. No plan in this table is verified truck by truck: a volumetric column carries no layout.

	compact SCPTOP				BP-3D-SCPTOP (volumetric oracle)					
Instance	z	bound	trucks	t (s)	z	bound (cert.)	trucks	t (s)	Δz (%)	tighter bound
$I_8_5_2_90$	18 796.96 OPT	18 796.96	12	0.198	18 796.96^{t}	18 795.56	12	3064	0.0000	18 796.96 (compact)
$I_8_10_2_90_k25$	38 855.65^{m}	38 851.93	26	3269	38 853.17^{t}	38 791.19	26	3062	−0.0064	38 851.93 (compact)
$I_16_15_2_90_k25$	68 139.17 OPT	68 139.17	45	1350	68 139.21^{t}	68 131.48	45	3117	+0.0001	68 139.17 (compact)
$I_16_15_2_90_k50$	134 346.91^{m}	133 417.47	89	1122	134 507.47^{t}	133 484.56	89	3073	+0.1195	133 484.56 (DW)
$I_24_20_2_90_k50$	200 266.21^{m}	199 518.04	137	2093	200 048.69^{t}	200 013.57	137	3071	−0.1086	200 013.57 (DW)
$I_24_20_2_90_k80$	282 101.77^{m}	280 823.53	189	2403	281 233.28^{t}	280 910.22	188	3133	−0.3079	280 910.22 (DW)

The volumetric problem of Appendix A earns its place in this study for one reason: it is the only part of the problem for which an independent optimum exists, because a compact formulation of it can be solved exactly on the smaller instances. What Table 7 measures is therefore not which method wins but whether the procedure arrives where it should on a case whose answer is known. Both sides were given the same 3 600 s, 20 GB and ten threads.

On the two instances the compact model closes, it does. On $I_8_5_2_90$ the procedure returns 18 796.96 with twelve trucks — the proved optimum, to the cent, reached from a different formulation by a different search. On $I_16_15_2_90_k25$ it returns 68 139.21 against a proved 68 139.17, also at forty-five trucks: four cents on sixty-eight thousand, 0.0001% above the truth, with its own certified bound 0.0113% below it. The procedure brackets a known optimum from both sides to within about a hundredth of a percent. That is the check this subsection exists to report, and it passes.

Where the compact model works it is also far the faster, at 0.2 s and 1 350 s against the procedure's roughly 3 070 s. On a volumetric problem small enough to write down whole, writing it down whole is the better plan; the decomposition is built for the problem the compact model cannot express, and Section 9.3 is where that is asked of it.

On the four instances where the compact model exhausts its memory cap with its dual bound short of its incumbent, the two agree closely enough to say the procedure is solving the same problem well: costs within 0.31% on every row, the same fleet on three and one truck fewer on $I_24_20_2_90_k80$. The procedure's plan is cheaper on three and dearer on one, by 0.1195% on $I_16_15_2_90_k50$, the row where it opened twelve

nodes in the hour and left its own interval 0.76% wide. At this accuracy the two are measuring each other, and neither row is a ranking.

Where the decomposition does separate itself is the bound: on the three largest instances its certificate is the tighter of the two, by 67, 496 and 87 cost units. The compact model's dual bound degrades as its memory cap approaches, while the master LP over a finite pool certifies whatever the pool supports.

One such statement needs no comparison of plans at all. On $I_12_12_2_90_k25$ the method certifies 52 105.31 — a full sweep in which every pair priced optimal and nothing priced out, over the volumetric relaxation and so valid for the 3D problem by Proposition 2 — 274 s into its run, where the compact volumetric model still stood at a dual bound of 51 944.22 when its 600 s expired, both at ten threads and a 20 GB cap. Both bound the same relaxation, and the decomposition's is 0.310% tighter.

9.3 What the 3D side gives

Table 8: BP-3D-SCPTOP on the size ladder, every row from one source tree (e4830fe) run alone on one machine, ten threads, a 20 GB per-model cap (1 GB on volumetric pricing, Table 5) and no seed set, with the rows (88)–(89) in steps 2 and 3 and step 4 limited to one sixth of the budget per call. *root* is the plan the construction returns before any node is popped, with the time it arrived, and *tree* the improvement the search then makes on it, relative to the root plan's cost; both are read from the same run's trace, so what branching contributes is measured within a run rather than across runs. *open* and *dropped* are as in Table 6; on row 4 four of the seven open regions carry no bound of their own. No row ends with both at 0, and both at 0 would not mean the space was refuted either: a region closed with a certified bound below the incumbent is bounded and not excluded, and row 1 ends with three such regions, which is why it claims no optimum of its own. The bound is the certified volumetric relaxation, valid for 3D by Proposition 2; it is loose wherever the crush limit binds, so the gap on the last two rows is not an optimality gap. LB and *gap* are the *method's own* certificate on every row, never a mixture: on rows 2 and 4 the compact volumetric model supplies a tighter valid bound (38 851.93 at a memory stop, 68 139.17 proved optimal), which Table 7 reports and Table 9 uses as an interval end. Row 1 is the paper's only proved optimum and the proof is not the method's: the compact volumetric relaxation of that instance solves to 18 796.96 in 0.198 s and this verified 3D plan attains it. The wall clock stops short of the budget by construction, 15% of it being reserved for the frontier sweep. The twelve columns are set as two blocks of the same five rows in the same order, so that the table needs no reduction in type size.

Instance	z	trucks	verified	LB (cert.)	gap
$I_8_5_2_90$	**18 796.96**	12	12 / 12	18 795.56	0.0075%
$I_8_10_2_90_k25$	38 854.44	26	26 / 26	38 791.19	0.163%
$I_12_12_2_90_k25$	52 116.76	34	34 / 34	52 105.31	0.022%
$I_16_15_2_90_k25$	79 703.31	55	55 / 55	68 131.48	14.5%
$I_24_20_2_90_k50$	223 944.20	156	156 / 156	200 013.57	10.7%

Instance	root (t)	tree	nodes	dropped	open	wall / budget (s)
$I_8_5_2_90$	18 797.59 (142 s)	0.003%	301	1	6	765 / 900
$I_8_10_2_90_k25$	38 867.74 (154 s)	0.034%	15	2	9	770 / 900
$I_12_12_2_90_k25$	52 215.42 (298 s)	0.19%	24	5	6	1 548 / 1 800
$I_16_15_2_90_k25$	92 251.99 (948 s)	**13.6**%	2	0	7	1 534 / 1 800
$I_24_20_2_90_k50$	223 944.20 (2 772 s)	0%	1	1	2	3 094 / 3 600

The size ladder takes the method, in the configuration of Sections 7 and 8, to twenty periods and 156 trucks, every plan verified truck by truck and every gap taken against a certified bound. On the first three rungs the truck count is in addition the minimum possible: each run prunes the region with one truck fewer by the requirement floor of Section 8, arithmetic over volume, payload and unit count and so valid under 3D loading. What remains of those gaps, 0.0075%–0.163%, is inventory cost. The jump to 14.5% and 10.7% on the last two rungs is attributed chiefly to the bound, on the mechanistic evidence of Section 9.5; nothing here proves that attribution or measures its share, and those two plans are reported as verified upper

bounds under a stated budget. The claim about the gaps is therefore conditional: they are small where the volumetric relaxation is nearly exact, which is checkable in advance.

What branching contributes is uneven and is not averaged away. Read within one run (Table 8), the tree improves the root construction by 13.6% on the fourth rung, where two level-1 nodes take a 67-truck root plan at 92 251.99 to 65 trucks and then to 55 at 79 703.31; by 0.19%, 0.034% and 0.003% on the third, second and first; and not at all on the fifth, whose root plan arrived, after one failed construction, under 290 s before the search window closed: one node, in which the region with fewer trucks returned no plan. The pattern follows what the root leaves the tree: where the relaxation is nearly exact the construction is already there, and where it is not the region with one truck fewer is where the plan improves.

The bound is cheap, which makes the division of labour of Section 6.4 affordable: on the five ladder rungs the volumetric certificate takes 1.3 to 13.8 s once root column generation stops, every pair proved, while the geometric sweep certified no region on any run. Nor do the placeability tests take the time on the ladder: they cost 0.2 to 14.2 s per run, while column generation, geometric pricing included, spends the budget share it is given, and on the last two rungs the construction takes most of the rest — the fourth rung's root construction alone took 669 s, and the fifth's two root attempts 919 and 938 s, the first ending when step 4 used its whole 600 s. None of the ladder's 8 335 placeability calls was left undecided: the shelf design placed 6 749 loads, and the exact test of Section 8.3 placed 485 and proved the other 1 101 unplaceable, none by the arithmetic conditions alone, which the partition now carries as rows. On the fifth rung one run isolates those rows, the method without them at the same step-4 limit and otherwise the same code: 156 verified trucks against 160, and 913 of 5 700 compositions submitted to the oracle proved unplaceable against 1 440 of 6 812. One run each; the cost, 2.6% lower relative to the run without the rows, is not claimed.

9.4 The compact model

Table 9: The compact 3D-SCPTOP model (a catalogue of $n_{\mathrm{LC}} = 3$ layouts shared by the trucks of a direction, 1 800 s, Gurobi 12) against BP-3D-SCPTOP on the size ladder, the method's rows being those of Table 8. **The budgets differ on three rows**: the compact model had 1 800 s throughout, the method 900 s on the first two rows, 1 800 s on the next two — which are therefore the equal-budget pairs — and 3 600 s on the last. *none* means no feasible solution was found inside the budget, and all three such runs stopped on *time* with peak resident sets of 6.1, 2.7 and 4.5 GB against the declared 20 GB cap. The last column is the interval in which $z^\star$ lies, taking the method's plan as the upper end and, as the lower end, the tighter bound *that bounds the problem*: $n_{\mathrm{LC}} = 3$ is an a-priori restriction of Section 4, so the compact 3D model's dual bound bounds that restriction and not $z^\star$ and appears in its own column only, while a compact *volumetric* dual bound is valid for 3D by inclusion and is used instead where one exists. Row 1 is the one optimum (Table 8).

	compact 3D-SCPTOP, 1 800 s				BP-3D-SCPTOP					
Instance	z	trucks	LB	stop	z	trucks	LB (cert.)	gap	budget (s)	$z^\star$ in
$I_8_5_2_90$	18 816.13	12	18 795.76	TIME	18 796.96	12	18 795.56	0.0075%	900	[18 796.96, 18 796.96]
$I_8_10_2_90_k25$	*none*	—	36 458.75	TIME	38 854.44	26	38 791.19	0.163%	900	[38 851.93, 38 854.44]
$I_12_12_2_90_k25$	*none*	—	48 516.86	TIME	52 116.76	34	52 105.31	0.022%	1 800	[52 105.31, 52 116.76]
$I_16_15_2_90_k25$	*none*	—	64 053.23	TIME	79 703.31	55	68 131.48	14.5%	1 800	[68 139.17, 79 703.31]
$I_24_20_2_90_k50$	not run				223 944.20	156	200 013.57	10.7%	3 600	[200 013.57, 223 944.20]

A compact formulation of 3D-SCPTOP cannot serve as a yardstick beyond five periods. Written with a catalogue of $n_{\mathrm{LC}} = 3$ layouts shared by the trucks of a direction — a *restriction* of Section 4 chosen a priori, so its dual bound bounds the restriction and not $z^\star$ — it returns a plan on one of four ladder instances. On $I_8_5_2_90$, at 67 674 variables and 92 092 constraints, it stops on its 1 800 s limit with no phantom freight and twelve loads that pass the independent verifier, 0.102% above the method's plan at twice the method's budget. From ten periods on, at 119 424 to 322 734 variables, it returns *no feasible solution at all* in 1 800 s, all three stopping on time and not on memory — peak resident sets of 6.1, 2.7 and 4.5 GB against a declared 20 GB cap. Four rungs are four rungs; beyond five periods the compact model supplies neither a plan nor a bound that bounds the problem.

$I_8_5_2_90$ is also the one instance on which this paper reports an optimum, and the proof is not the method's. The compact *volumetric* relaxation solves it to proved optimality at 18 796.96 in 0.198 s, every placeable load is volumetrically feasible, and the verified 3D plan attains that value; the method's own search does not establish it: the run of Table 8 ends with one region dropped, six open and three covered by certified volumetric bounds of 18 795.56, 18 795.54 and 18 795.54, below the incumbent and so bounded

Table 10: What volumetric planning costs, constructed rather than estimated: trucks dispatched on the two instances where the per-stack crush limit binds. Row 1 is the volumetric benchmark's own plan, which is not executable — 41 of its 45 dispatched loads on the first instance and 119 of its 137 on the second are *proved* unplaceable with *none* left undecided, so those shares carry no undecided caveat. Row 2 splits each unplaceable load where it stands, with no consolidation; it is reported for contrast and is *not* the comparison, consolidation being worth twelve and forty-four trucks. Row 3 is the comparison: the volumetric plan's delivery quantities kept, each period re-packed with the partition-and-pack machinery of the first implementation, which lacks the rows (88)–(89), every resulting truck verified, and at least the original quantities delivered — by 80 and 137 units, the partition being a cover. Row 4 is BP-3D-SCPTOP, which also chooses the quantities. The volumetric plan on $I_24_20_2_90_k50$ is a *memory stop* and not an optimum, so a better one might repair into fewer than 212 trucks and that column's margin is overstated by an unquantified amount in the method's favour; the $I_16_15_2_90_k25$ plan is the proved volumetric optimum and carries no such caveat, though a re-pack with the method's current partition might need fewer trucks on either instance. Row 3's cost is the volumetric plan's plus cK per extra truck; it omits the holding cost of the over-delivered units, which could only raise it.

	$I_16_15_2_90_k25$	$I_24_20_2_90_k50$
volumetric plan (not executable)	45	137
repaired load by load (contrast only)	86	256
re-packed, quantities kept	**74**	**212**
BP-3D-SCPTOP (which also chooses the quantities)	**55**	**156**
saving on trucks / on cost	25.7% / 17.9%	26.4% / 18.6%

rather than refuted. The proof is available because this instance's two container families have coinciding height- and crush-limited layer counts — exactly the condition under which Section 9.5 finds a volumetric capacity adequate.

9.5 Is a volumetric capacity adequate?

What a volumetric capacity costs is constructed rather than estimated (Table 10): each volumetric plan's delivery quantities are kept and each period's freight re-packed with the partition-and-pack machinery of the first implementation, every resulting truck verified. The re-packed plans need 74 and 212 trucks where the method plans 55 and 156 directly — savings of 25.7% and 26.4% on trucks and 17.9% and 18.6% on cost, each denominator the re-packed plan's own figure. Two caveats travel with them: the re-packed plan delivers *at least* the volumetric quantities, by 80 and 137 units, the partition being a cover; and the $I_24_20_2_90_k50$ volumetric plan is a memory stop rather than an optimum, so a better one might repair into fewer than 212 trucks and that row's margin is overstated in the method's favour. The $I_16_15_2_90_k25$ row, the proved optimum, carries no such caveat.

Deflating the capacity is the obvious remedy, and one step is measured: adding the stack-count rows (87) to the compact volumetric model, valid for every placeable load, raises its answer on $I_16_15_2_90_k25$ from forty-five trucks to forty-eight and its dual bound to 70 435.04 after 600 s — 2 296 above that model's own bound of 68 139.17 — while changing nothing where the crush limit does not bind. Whether those forty-eight loads can be placed has not been tested, the stack rows being necessary for placeability and not sufficient, so the stack-aware model is not yet a baseline.

The mechanism, and a test a planner can run. The over-credit has one cause, measured directly on $I_16_15_2_90_k25$: the payload admits 57 full RTIs of product P_1 in one truck while 32 is the largest number that can be placed, the placeability cap the method's run bisects to proof before its search starts. The container is 650 mm tall, so four layers fit under the 3 000 mm roof, while P_1 at 419 kg full fits only two under the 1 200 kg crush limit, as do six of the family's eight products. A volumetric fill therefore credits a truck with roughly twice that family's placeable count, which is also why the certified bound on the two crush-binding rungs sits 14.5% and 10.7% below the method's plan.

That mechanism can be checked before anything is solved, since it needs only the catalogue and the trailer. For one container type, let the *analytic credit ratio* κ be the count a volumetric capacity credits over

an *analytic* count of how many can be carried,

$$\kappa_r = \frac{\min\{\Gamma_K/\gamma_{pr},\ pK^w/\bar{w}_r\}}{\min\{s_r \min(L_r^h, L_r^c),\ pK^w/\bar{w}_r\}}, \qquad L_r^h = \left\lfloor \frac{rK}{r_{pr}} \right\rfloor, \quad L_r^c = \left\lfloor \frac{W^{\max}}{\bar{w}_r} \right\rfloor, \tag{90}$$

with $\bar{w}_r$ the catalogued gross weight of one full RTI and s_r the number of its footprints a single orientation tiles onto the floor. The denominator is a grid count with no packing oracle run, hence an *upper* estimate of what fits, so $\kappa > 1$ is a conservative diagnosis of over-credit. Over the fifteen types of the industrial catalogue κ has median 1.000, ranges to 2.000 and exceeds 1.20 on four types, and *no* type has $\kappa < 1$: the error cannot run in the planner's favour, and a plan over a mixed catalogue fails on its worst container. The instances here are *generated*, with κ in a range the generator controls, so this is a realism check on that range rather than a claim that their results transfer to a particular plant.

10 Conclusions

Procurement transport planning with returnable transport items has been formulated with transport capacity as what it physically is — a three-dimensional loading condition on every dispatched truck — and solved by branch and price over complete loaded trucks. Because 3D-SCPTOP is written over trucks it is block-angular, so the Dantzig–Wolfe master is obtained by aggregation and carries no loading configuration (Table 4), and the equivalence needs no hypothesis. Every lower bound reported rests on Proposition 2, which makes a bound affordable where a geometric certificate is not. And a volumetric transport capacity is not a conservative simplification: the method needs 25.7% and 26.4% fewer trucks than two volumetric plans' quantities need once re-packed into loadable trucks, with the first implementation's earlier partition (Table 10), and on the industrial catalogue the error runs one way only.

Three limits are real, and each names its next step. The method certifies no optimum here: the search ends with regions unrefuted, and the one optimum reported is established by inclusion from an exactly solved relaxation. The bound is loose exactly where the crush limit halves a container family's layer count, and the stack-count rows (87), which the construction already carries, tighten it measurably when added to the compact volumetric relaxation, so they and the finer rows (88)–(89), which the volumetric relaxation does not imply, are the candidates for tightening the bound inside the search as well, once it is known whether the loads they admit can be placed. Closing the regions the search leaves unrefuted may need a branching rule on *pairs* of columns, untested here. And the comparison against a procedure in industrial use is to be run on instances a plant solves itself, against a plan supplied to it.

A The SCPTOP benchmark

The volumetric benchmark extends the procurement transport model of (Díaz-Madroñero, Mula, and Peidro 2014) to the closed-loop two-node setting studied here: fuzzy parameters are dropped, transport capacity is expressed through the volume of the individual RTIs against the truck volume rather than a maximum container count, and a product may travel in more than one RTI type. To isolate the effect of the loading representation it is identical to the transport and inventory block of 3D-SCPTOP in every other respect — same two nodes, closed loop, lead time, coverage stock, payload cap, full-truckload floor and objective — and differs in one place: where 3D-SCPTOP asks whether the RTIs can be stacked and placed on the floor, SCPTOP asks only whether their volumes sum to less than the trailer's. The notation is that of Section 4, with $\overline{V}_{knt}$ a decision variable here rather than the slot sum (3).

$$\text{Min } z = \sum_{g\in G}\sum_{k\in K}\sum_{t\in T} cK\, Y_{gkt} + \sum_{g\in G}\sum_{n\in N}\sum_{t\in T} ic_{gn} I_{gnt} \tag{91}$$

subject to, for all $k \in K$ and $t \in T$,

$$\sum_{n\in N} \gamma_{p\rho(n)} \overline{V}_{knt} \le \Gamma_K\, Y_{pkt}, \qquad \sum_{r\in R} \gamma_{sr} V^{\mathrm{e}}_{krt} \le \Gamma_K\, Y_{skt} \tag{92}$$

$$\sum_{n\in N} w_n \overline{V}_{knt} \le pK^w Y_{pkt}, \qquad \sum_{r\in R} w^{\mathrm{f}}_r V^{\mathrm{e}}_{krt} \le pK^w Y_{skt} \tag{93}$$

together with the full-truckload floor on the binding resource, imposed exactly as in (52),

$$\sum_{n} \gamma_{p\rho(n)} \overline{V}_{knt} \ \ge\ \alpha\Gamma_K \big(Y_{pkt} - 1 + \theta_{pkt}\big), \qquad \sum_{n} w_n \overline{V}_{knt} \ \ge\ \alpha\, pK^w \big(Y_{pkt} - \theta_{pkt}\big), \tag{94}$$

with $\theta_{gkt} \in \{0, 1\}$ and the supplier direction analogous. The inventory balances, coverage stock and domains are (8)–(19) unchanged, and (45) is carried here too, so neither compact model is held to an easier standard. Constraint (92) is the whole of the benchmark's loading model: no footprint, stack height, layer count or crush limit appears.

References

Ali, Sara et al. (June 2022). "On-line three-dimensional packing problems: A review of off-line and on-line solution approaches". In: *Computers and Industrial Engineering* 168. ISSN: 03608352. DOI: 10.1016/j.cie.2022.108122.

Alonso, M. T., R. Alvarez-Valdes, and F. Parreño (Mar. 2020). "A GRASP algorithm for multi container loading problems with practical constraints". In: *4OR* 18.1, pp. 49–72. ISSN: 16142411. DOI: 10.1007/s10288-018-0397-z.

Alonso, M. T., A. Martinez-Sykora, et al. (Nov. 2022). "The pallet-loading vehicle routing problem with stability constraints: The pallet-loading vehicle routing problem with stability constraints. Second revision". In: *European Journal of Operational Research* 302.3, pp. 860–873. ISSN: 03772217. DOI: 10.1016/j.ejor.2022.01.035.

Baller, Reinhard et al. (July 2022). "Optimizing automotive inbound logistics: A mixed-integer linear programming approach". In: *Transportation Research Part E: Logistics and Transportation Review* 163. ISSN: 13665545. DOI: 10.1016/j.tre.2022.102734.

Barnhart, Cynthia et al. (1998). "Branch-and-Price: Column Generation for Solving Huge Integer Programs". In: *Operations Research* 46.3, pp. 316–329. DOI: 10.1287/opre.46.3.316.

Beasley, J. E. (1985). "An Exact Two-Dimensional Non-Guillotine Cutting Tree Search Procedure". In: *Operations Research* 33.1, pp. 49–64. DOI: 10.1287/opre.33.1.49.

Berman, Oded and Qian Wang (2006). "Inbound logistic planning: Minimizing transportation and inventory cost". In: *Transportation Science* 40.3, pp. 287–299. ISSN: 15265447. DOI: 10.1287/trsc.1050.0130.

Bortfeldt, Andreas and Gerhard Wäscher (Aug. 2013). "Constraints in container loading-A state-of-the-art review". In: *European Journal of Operational Research* 229.1, pp. 1–20. ISSN: 03772217. DOI: 10.1016/j.ejor.2012.12.006.

Chen, C S, S M Lee, and Q S Shen (1995). *EUROPEAN JOURNAL OF OPERATIONAL RESEARCH An analytical model for the container loading problem.* Tech. rep., pp. 68–76.

Chen, Mingzhou, Jiazhen Huo, and Yongrui Duan (June 2023). "A hybrid biogeography-based optimization algorithm for three-dimensional bin size designing and packing problem". In: *Computers and Industrial Engineering* 180. ISSN: 03608352. DOI: 10.1016/j.cie.2023.109239.

Chitsaz, Masoud, Jean François Cordeau, and Raf Jans (Dec. 2019). "A unified decomposition matheuristic for assembly, production, and inventory routing". In: *INFORMS Journal on Computing* 31.1, pp. 134–152. ISSN: 15265528. DOI: 10.1287/ijoc.2018.0817.

Chopra, Sunil and Peter Meindl (2015). *Supply chain management : strategy, planning, and operation.* ISBN: 9780133800203.

Christofides, Nicos and Charles Whitlock (1977). "An Algorithm for Two-Dimensional Cutting Problems". In: *Operations Research* 25.1, pp. 30–44. DOI: 10.1287/opre.25.1.30.

Clautiaux, François, Cláudio Alves, and José Valério de Carvalho (2010). "A Survey of Dual-Feasible and Superadditive Functions". In: *Annals of Operations Research* 179.1, pp. 317–342. DOI: 10.1007/s10479-008-0453-8.

Dantzig, George B. and Philip Wolfe (1960). "Decomposition Principle for Linear Programs". In: *Operations Research* 8.1, pp. 101–111. DOI: 10.1287/opre.8.1.101.

Desaulniers, Guy, Jacques Desrosiers, and Marius M. Solomon, eds. (2005). *Column Generation.* Boston, MA: Springer. DOI: 10.1007/b135457.

Díaz-Madroñero, Manuel, Josefa Mula, and David Peidro (Sept. 2014). *A review of discrete-time optimization models for tactical production planning.* DOI: 10.1080/00207543.2014.899721.

— (Dec. 2017). "A mathematical programming model for integrating production and procurement transport decisions". In: *Applied Mathematical Modelling* 52, pp. 527–543. ISSN: 0307904X. DOI: 10.1016/j.apm.2017.08.009.

Díaz-Madroñero, Manuel, David Peidro, and Josefa Mula (Dec. 2014). "A fuzzy optimization approach for procurement transport operational planning in an automobile supply chain". In: *Applied Mathematical Modelling* 38.23, pp. 5705–5725. ISSN: 0307904X. DOI: 10.1016/j.apm.2014.04.053.

— (Oct. 2015). *A review of tactical optimization models for integrated production and transport routing planning decisions*. DOI: 10.1016/j.cie.2015.06.010.

Dominguez, Oscar et al. (Jan. 2016). "Using biased randomization for solving the two-dimensional loading vehicle routing problem with heterogeneous fleet". In: *Annals of Operations Research* 236.2, pp. 383–404. ISSN: 15729338. DOI: 10.1007/s10479-014-1551-4.

Dyckhoff, Harald (1990). *A typology of cutting and packing problems*. Tech. rep., pp. 145–159.

Fekete, Sándor P. and Jörg Schepers (2001). "New Classes of Fast Lower Bounds for Bin Packing Problems". In: *Mathematical Programming* 91.1, pp. 11–31. DOI: 10.1007/s101070100243.

— (2004). "A General Framework for Bounds for Higher-Dimensional Orthogonal Packing Problems". In: *Mathematical Methods of Operations Research* 60.2, pp. 311–329. DOI: 10.1007/s001860400376.

Fleischmann, Bernhard, Herbert Kopfer, and Christopher Sürie (Mar. 2015). "Transport Planning for Procurement and Distribution". In: *Supply Chain Management and Advanced Planning*. Ed. by Hartmut Stadtler, Christoph Kilger, and Herbert Meyr. Springer Texts in Business and Economics. Springer. Chap. 12, pp. 225–240. DOI: 10.1007/978-3-642-55309-7. URL: https://ideas.repec.org/h/spr/sptchp/978-3-642-55309-7_12.html.

Gonçalves, José Fernando and Mauricio G.C. Resende (Oct. 2013). "A biased random key genetic algorithm for 2D and 3D bin packing problems". In: *International Journal of Production Economics* 145.2, pp. 500–510. ISSN: 09255273. DOI: 10.1016/j.ijpe.2013.04.019.

Guo, Min et al. (2024). "Incentivising-by-penalty: The optimal return strategy for a reusable transport item rental platform". In: *Transportation Research Part D: Transport and Environment* 134, p. 104339. ISSN: 1361-9209. DOI: https://doi.org/10.1016/j.trd.2024.104339. URL: https://www.sciencedirect.com/science/article/pii/S1361920924002967.

Harrath, Youssef (Sept. 2022). "A three-stage layer-based heuristic to solve the 3D bin-packing problem under balancing constraint". In: *Journal of King Saud University - Computer and Information Sciences* 34 (8), pp. 6425–6431. ISSN: 22131248. DOI: 10.1016/j.jksuci.2021.07.007.

Hashemi-Amiri, Omid, Fahimeh Ghorbani, and Ran Ji (Jan. 2023). "Integrated supplier selection, scheduling, and routing problem for perishable product supply chain: A distributionally robust approach". In: *Computers and Industrial Engineering* 175. ISSN: 03608352. DOI: 10.1016/j.cie.2022.108845.

Hein, Fanny and Christian Almeder (July 2016). "Quantitative insights into the integrated supply vehicle routing and production planning problem". In: *International Journal of Production Economics* 177, pp. 66–76. ISSN: 09255273. DOI: 10.1016/j.ijpe.2016.04.014.

Junqueira, Leonardo, Reinaldo Morabito, and Denise Sato Yamashita (2012a). "MIP-based approaches for the container loading problem with multi-drop constraints". In: *Annals of Operations Research* 199.1, pp. 51–75. ISSN: 15729338. DOI: 10.1007/s10479-011-0942-z.

— (Jan. 2012b). "Three-dimensional container loading models with cargo stability and load bearing constraints". In: *Computers and Operations Research* 39.1, pp. 74–85. ISSN: 03050548. DOI: 10.1016/j.cor.2010.07.017.

Junqueira, Leonardo, José F. Oliveira, et al. (Sept. 2013). "An optimization model for the vehicle routing problem with practical three-dimensional loading constraints". In: *International Transactions in Operational Research* 20.5, pp. 645–666. ISSN: 09696016. DOI: 10.1111/j.1475-3995.2012.00872.x.

Kurpel, Deidson Vitorio et al. (July 2020). "The exact solutions of several types of container loading problems". In: *European Journal of Operational Research* 284.1, pp. 87–107. ISSN: 03772217. DOI: 10.1016/j.ejor.2019.12.012.

Langley, C. John et al. (2020). *Supply Chain Management: A Logistics Perspective*. 11th. Cengage Learning, pp. 413–452.

Li, Ying, Mingzhou Chen, and Jiazhen Huo (Mar. 2022). "A hybrid adaptive large neighborhood search algorithm for the large-scale heterogeneous container loading problem". In: *Expert Systems with Applications* 189. ISSN: 09574174. DOI: 10.1016/j.eswa.2021.115909.

Mahvash, Batoul, Anjali Awasthi, and Satyaveer Chauhan (2018). "A column generation-based heuristic for the three-dimensional bin packing problem with rotation". In: *Journal of the Operational Research Society* 69 (1), pp. 78–90. ISSN: 14769360. DOI: 10.1057/s41274-017-0186-7.

Meyer, Anne and Boris Amberg (May 2018). "Transport concept selection considering supplier milk runs – An integrated model and a case study from the automotive industry". In: *Transportation Research Part E: Logistics and Transportation Review* 113, pp. 147–169. ISSN: 13665545. DOI: 10.1016/j.tre.2017.07.004.

Mosca, Alyssa, Navneet Vidyarthi, and Ahmet Satir (Jan. 2019). *Integrated transportation – inventory models: A review*. DOI: 10.1016/j.orp.2019.100101.

Murty, Katta G. (1968). "An Algorithm for Ranking all the Assignments in Order of Increasing Cost". In: *Operations Research* 16.3, pp. 682–687. DOI: 10.1287/opre.16.3.682.

Paquay, C, M Schyns, and S Limbourg (2016). "INTERNATIONAL TRANSACTIONS IN OPERATIONAL RESEARCH A mixed integer programming formulation for the three-dimensional bin packing problem deriving from an air cargo application". In: *Intl. Trans. in Op. Res* 23, pp. 187–213. DOI: 10.1111/itor.12111. URL: https://onlinelibrary.wiley.com/doi/10.1111/itor.12111.

Que, Quanqing, Fang Yang, and Defu Zhang (Mar. 2023). "Solving 3D packing problem using Transformer network and reinforcement learning". In: *Expert Systems with Applications* 214. ISSN: 09574174. DOI: 10.1016/j.eswa.2022.119153.

Sancak, Emre and F. Sibel Salman (June 2011). "Multi-item dynamic lot-sizing with delayed transportation policy". In: *International Journal of Production Economics* 131.2, pp. 595–603. ISSN: 09255273. DOI: 10.1016/j.ijpe.2011.02.004.

Schöneberg, Tim, Achim Koberstein, and Leena Suhl (Dec. 2010). "An optimization model for automated selection of economic and ecologic delivery profiles in area forwarding based inbound logistics networks". In: *Flexible Services and Manufacturing Journal* 22.3-4, pp. 214–235. ISSN: 19366582. DOI: 10.1007/s10696-011-9084-5.

— (Nov. 2013). "A stochastic programming approach to determine robust delivery profiles in area forwarding inbound logistics networks". In: *OR Spectrum* 35.4, pp. 807–834. ISSN: 01716468. DOI: 10.1007/s00291-013-0349-0.

Silva, Everton Fernandes, Túlio Angelo Machado Toffolo, and Tony Wauters (Sept. 2019). "Exact methods for three-dimensional cutting and packing: A comparative study concerning single container problems". In: *Computers and Operations Research* 109, pp. 12–27. ISSN: 03050548. DOI: 10.1016/j.cor.2019.04.020.

Soysal, Mehmet et al. (2023). "Managing returnable transport items in a vendor managed inventory system". In: *Socio-Economic Planning Sciences* 86, p. 101504. ISSN: 0038-0121. DOI: https://doi.org/10.1016/j.seps.2022.101504. URL: https://www.sciencedirect.com/science/article/pii/S0038012122003111.

Stadtler, Hartmut, Christoph Kilger, and Herbert Meyr (2015). *Supply Chain Management and Advanced Planning*. Vol. 5th Edition. URL: http://www.springer.com/series/10099.

Tresca, Giulia et al. (July 2022). "Automating Bin Packing: A Layer Building Matheuristics for Cost Effective Logistics". In: *IEEE Transactions on Automation Science and Engineering* 19 (3), pp. 1599–1613. ISSN: 15583783. DOI: 10.1109/TASE.2022.3177422.

Vanderbeck, François (2000). "On Dantzig–Wolfe Decomposition in Integer Programming and Ways to Perform Branching in a Branch-and-Price Algorithm". In: *Operations Research* 48.1, pp. 111–128. DOI: 10.1287/opre.48.1.111.12453.

Wäscher, Gerhard, Heike Haußner, and Holger Schumann (Dec. 2007). "An improved typology of cutting and packing problems". In: *European Journal of Operational Research* 183.3, pp. 1109–1130. ISSN: 03772217. DOI: 10.1016/j.ejor.2005.12.047.

Wei, Lijun et al. (June 2015). "A variable neighborhood search for the capacitated vehicle routing problem with two-dimensional loading constraints". In: *European Journal of Operational Research* 243.3, pp. 798–814. ISSN: 03772217. DOI: 10.1016/j.ejor.2014.12.048.

Zhang, Yipei et al. (2024). "Closed-loop inventory routing problem for perishable food with returnable transport items selection". In: *International Journal of Production Research* 62.1-2, pp. 501–521. DOI: 10.1080/00207543.2023.2275639. eprint: https://doi.org/10.1080/00207543.2023.2275639. URL: https://doi.org/10.1080/00207543.2023.2275639.

Zhao, Hang, Chenyang Zhu, and Hui Huang (2022). "Learning practically feasible policies for online 3D bin packing". In: 65.112105, p. 17. DOI: 10.1007/s11432-021-3348-6. URL: https://doi.org/10.1007/s11432-021-3348-6.

Zhao, Xiaozhou et al. (2016). "INTERNATIONAL TRANSACTIONS IN OPERATIONAL RESEARCH A comparative review of 3D container loading algorithms". In: *Intl. Trans. in Op. Res* 23, pp. 287–320. DOI: 10.1111/itor.12094. URL: https://onlinelibrary.wiley.com/doi/10.1111/itor.12094.

Zhu, Wenbin and Andrew Lim (Nov. 2012). "A new iterative-doubling Greedy-Lookahead algorithm for the single container loading problem". In: *European Journal of Operational Research* 222.3, pp. 408–417. ISSN: 03772217. DOI: 10.1016/j.ejor.2012.04.036.